Recursion

1. Recursive Implementation of `atoi()`

You are given a string `s` that may contain leading spaces, an optional sign (`+` or `-`), digits, and other characters.
Your task is to convert this string into a **32-bit signed integer**, exactly like `atoi()`, using **recursion**.

Rules to follow:

- Ignore leading whitespaces
- Check and apply sign (`+` or `-`)
- Read digits until a non-digit character is found
- Stop conversion at the first non-digit
- Clamp the result within `[-2147483648, 2147483647]`

Example
Input: `"  -12345"`
Output: `-12345`

Approach

We convert the string to an integer using recursion while following these steps:

1. Skip leading whitespaces.
2. Detect sign (`+` or `-`).
3. Recursively read digit characters and build the number.
4. Stop when a non-digit character is found.

5. Clamp the result within 32-bit signed integer limits.

Code (Recursive)

```
#include <bits/stdc++.h>
using namespace std;

int INT_MIN_VAL = -2147483648;
int INT_MAX_VAL = 2147483647;

// Recursive helper function
int solve(string &s, int idx, long long num, int sign) {
    // Stop if index out of range or character is not a digit
    if (idx >= s.size() || !isdigit(s[idx])) {
        return sign * num;
    }

    // Add current digit
    num = num * 10 + (s[idx] - '0');

    // Handle overflow
    if (sign * num <= INT_MIN_VAL) return INT_MIN_VAL;
    if (sign * num >= INT_MAX_VAL) return INT_MAX_VAL;

    // Recurse to next character
    return solve(s, idx + 1, num, sign);
}

int myAtoi(string s) {
    int i = 0;

    // Skip leading spaces
    while (i < s.size() && s[i] == ' ') i++;

    // Handle sign
    int sign = 1;
    if (i < s.size() && (s[i] == '+' || s[i] == '-')) {
```

```
        if (s[i] == '-') sign = -1;
        i++;
    }

    // Start recursive digit processing
    return solve(s, i, 0, sign);
}

int main() {
    string s1 = " -12345";
    string s2 = "4193 with words";

    cout << myAtoi(s1) << endl; // -12345
    cout << myAtoi(s2) << endl; // 4193

    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(n)$ — each character is processed once
- **Space Complexity:** $O(n)$ — recursion stack in the worst case

2. Implement Pow(x, n) | X raised to the power N

Explanation

You are given a base x (double) and an integer exponent n.
Your task is to compute x^n.

Important points:

- If n is **positive**, multiply x by itself n times
- If n is **negative**, result becomes `1 / (x^n)`
- If `n = 0`, the answer is always 1
- You must handle large values of n safely

Example

Input: `x = 2.0, n = -2`
Output: `0.25`
Explanation: `2^-2 = 1 / (2^2) = 1 / 4`

Brute Force Approach

Idea

Multiply x with itself `|n|` times.
If n is negative, first convert `x = 1/x` and make n positive.

Code (Brute Force)

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    double myPow(double x, int n) {
        if (n == 0) return 1.0;

        long long exp = n;
        if (exp < 0) {
            x = 1 / x;
            exp = -exp;
        }
```

```
        double ans = 1.0;
        for (long long i = 0; i < exp; i++) {
            ans *= x;
        }
        return ans;
    }
};

int main() {
    Solution sol;
    cout << sol.myPow(2.0, 10) << endl;
    cout << sol.myPow(2.0, -2) << endl;
    return 0;
}
```

Complexity

- Time: $O(n)$
- Space: $O(1)$

Optimal Approach (Fast Power / Binary Exponentiation)

Idea

Instead of multiplying x repeatedly, reduce the problem size:

- If n is even → $x^n = (x^2)^{(n/2)}$
- If n is odd → $x^n = x * x^{n-1}$

This reduces calculations drastically.

```
#include <bits/stdc++.h>
using namespace std;
```

```
class Solution {
private:
    double power(double x, long long n) {
        if (n == 0) return 1.0;

        double half = power(x, n / 2);

        if (n % 2 == 0)
            return half * half;
        else
            return half * half * x;
    }

public:
    double myPow(double x, int n) {
        long long exp = n;
        if (exp < 0)
            return 1.0 / power(x, -exp);

        return power(x, exp);
    }
};

int main() {
    Solution sol;
    cout << sol.myPow(2.0, 10) << endl;
    cout << sol.myPow(2.0, -2) << endl;
    return 0;
}
```

Complexity

- Time: $O(\log n)$
- Space: $O(\log n)$ (recursion stack)

3. Count Good Numbers

Explanation

A digit string of length n is called **good** if:

- Digits at **even indices (0-based)** are **even digits** → `{0, 2, 4, 6, 8}` → **5 choices**
- Digits at **odd indices** are **prime digits** → `{2, 3, 5, 7}` → **4 choices**

You must count how many such strings of length n exist.
Since the answer can be very large, return it modulo $10^9 + 7$.

Example

Input: `n = 2`
Index 0 → even → 5 choices
Index 1 → odd → 4 choices
Total = `5 × 4 = 20`

Key Observation

- Number of **even positions** = `(n + 1) / 2`
- Number of **odd positions** = `n / 2`

For each:

- Even positions → `5^(evenCount)`
- Odd positions → `4^(oddCount)`

Final Answer:

```
(5^(evenCount) * 4^(oddCount)) % MOD
```

To compute large powers efficiently, we use **Binary Exponentiation**.

Optimal Approach (Fast Power)

Code

```
#include <bits/stdc++.h>

using namespace std;


class Solution {

public:

    static const long long MOD = 1000000007;


    // Fast power function

    long long power(long long base, long long exp) {

        long long result = 1;

        base %= MOD;


        while (exp > 0) {

            if (exp & 1)

                result = (result * base) % MOD;


            base = (base * base) % MOD;

            exp >>= 1;

        }
```

```
        return result;
    }

    long long countGoodNumbers(long long n) {
        long long evenCount = (n + 1) / 2;
        long long oddCount = n / 2;

        long long evenWays = power(5, evenCount);
        long long oddWays = power(4, oddCount);

        return (evenWays * oddWays) % MOD;
    }
};

int main() {
    Solution sol;
    cout << sol.countGoodNumbers(1) << endl;
    cout << sol.countGoodNumbers(2) << endl;
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(log n)`
  (Binary exponentiation for power calculation)

- **Space Complexity:** `O(1)`
  (Only constant extra variables used)

4. Sort a Stack (Using Recursion)

Explanation
You are given a stack of integers and you need to sort it in **descending order**, meaning the **largest element should be on the top** of the stack.
You are **not allowed to use loops or any sorting algorithms**. Only **recursion** and basic stack operations (`push`, `pop`, `top`, `empty`) are allowed.

Idea
We solve this in two recursive steps:

1. **sortStack**
    - Remove the top element.
    - Recursively sort the remaining stack.
    - Insert the removed element back into the sorted stack at its correct position.
2. **insert**
    - Inserts an element into a stack that is already sorted in descending order.
    - If the stack is empty or the current top is smaller than or equal to the element, push it.

- Otherwise, pop the top, recursively insert the element, and then push the popped value back.

This way, the stack gets sorted when recursion unwinds.

Code

```
#include <bits/stdc++.h>
using namespace std;

// Inserts an element into the stack in descending order
void insert(stack<int>& s, int x) {
    // If stack is empty or correct position found
    if (s.empty() || s.top() <= x) {
        s.push(x);
        return;
    }

    int temp = s.top();
    s.pop();

    insert(s, x);

    s.push(temp);
}

// Sorts the stack using recursion
void sortStack(stack<int>& s) {
    // Base case
    if (s.empty()) return;

    int temp = s.top();
    s.pop();

    sortStack(s);

    insert(s, temp);
}
```

```
int main() {
    stack<int> s;
    s.push(4);
    s.push(1);
    s.push(3);
    s.push(2);

    sortStack(s);

    // Printing sorted stack (top to bottom)
    while (!s.empty()) {
        cout << s.top() << " ";
        s.pop();
    }

    return 0;
}
```

Example Dry Run (Input: 4 1 3 2)

- Stack is recursively emptied
- Elements are inserted back in correct descending position
- Final stack (top → bottom): 4 3 2 1

Complexity Analysis

- **Time Complexity:** $O(n^2)$
  Each insertion may take up to $O(n)$ and is done for n elements.
- **Space Complexity:** $O(n)$
  Due to recursion stack.

5. Reverse a Stack Using Recursion

Explanation
You are given a stack of integers and your task is to **reverse the stack in-place using recursion**.
You are **not allowed to use loops or any extra data structures** like arrays, vectors, or queues.
Only standard stack operations (`push`, `pop`, `top`, `empty`) and recursion are allowed.

Key Idea
To reverse a stack using recursion, we break the problem into two parts:

1. **reverseStack**
    - Remove the top element.
    - Recursively reverse the remaining stack.
    - Insert the removed element at the **bottom** of the stack.
2. **insertAtBottom**
    - Inserts a given element at the bottom of the stack using recursion.
    - If the stack is empty, push the element.
    - Otherwise, pop the top, recursively insert at bottom, then push the popped element back.

By repeating this process, the stack gets reversed when recursion unwinds.

Example

Input stack (top → bottom):
`[4, 1, 3, 2]`

After reversing:
`[2, 3, 1, 4]`

Code

```
#include <bits/stdc++.h>
using namespace std;

// Function to insert an element at the bottom of the stack
void insertAtBottom(stack<int> &st, int val) {
    // Base case: stack is empty
    if (st.empty()) {
        st.push(val);
        return;
    }

    int topVal = st.top();
    st.pop();

    insertAtBottom(st, val);

    st.push(topVal);
}

// Function to reverse the stack using recursion
void reverseStack(stack<int> &st) {
    // Base case
    if (st.empty()) return;

    int topVal = st.top();
    st.pop();

    reverseStack(st);

    insertAtBottom(st, topVal);
}

int main() {
    stack<int> st;
    st.push(4);
    st.push(1);
    st.push(3);
```

```
    st.push(2);

    reverseStack(st);

    // Print reversed stack (top to bottom)
    while (!st.empty()) {
        cout << st.top() << " ";
        st.pop();
    }

    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(n^2)$
  Each element is inserted at the bottom, which takes $O(n)$ time, for n elements.

- **Space Complexity:** $O(n)$
  Due to recursive call stack.

6. Generate All Binary Strings (No Consecutive 1s)

Explanation
You are given an integer n. You need to generate **all binary strings of length n** such that **no two 1s are adjacent**.
The result should be in **lexicographically increasing order**.

A binary string contains only characters `'0'` and `'1'`.

Key Observation

- At any position, you can **always place `'0'`**.
- You can place `'1'` **only if the previous character is not `'1'`**.
- Using recursion, we build the string character by character until its length becomes n.

Example

For `n = 3`, valid strings are:
`000, 001, 010, 100, 101`
Strings like `011` or `110` are not allowed because they contain consecutive 1s.

Approach

1. Use a recursive function with:
    - current string being built
    - result container
2. If the current string length becomes n, store it.
3. Always try adding `'0'`.
4. Add `'1'` only if the last character is not `'1'`.

Code

```
#include <bits/stdc++.h>
using namespace std;

void generate(int n, string curr, vector<string>& result) {
    // Base case: required length reached
    if (curr.length() == n) {
        result.push_back(curr);
        return;
```

```
    }

    // Always allowed to add '0'
    generate(n, curr + "0", result);

    // Add '1' only if previous character is not '1'
    if (curr.empty() || curr.back() != '1') {
        generate(n, curr + "1", result);
    }
}

int main() {
    int n = 3;
    vector<string> result;

    generate(n, "", result);

    for (string &s : result) {
        cout << s << " ";
    }

    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(2^n)`
  In the worst case, each position branches into two recursive calls.

- **Space Complexity:** `O(n)`
  Due to recursion depth (call stack).

7. Generate Parentheses

Explanation
You are given an integer n, representing the number of pairs of parentheses.
Your task is to generate **all possible combinations of well-formed parentheses** using exactly n opening ' (' and n closing ') ' brackets.

A parentheses string is **well-formed** if:

- At any point, the number of ') ' does not exceed ' (' .
- At the end, the total number of ' (' equals the number of ') ' .

Example
For n = 3, valid combinations are:

```
((()))
(()())
(())()
()(())
()()()
```

Brute Force Approach

Idea

Generate **all possible strings of length 2n** using ' (' and ') ' .
For each generated string, check whether it is valid using a balance counter.

Algorithm

1. Generate all sequences of length 2n.
2. For each sequence:
    - Traverse the string.

- Increase balance for `'('`, decrease for `')'`.
- If balance becomes negative at any point → invalid.
- At the end, balance must be `0`.

3. Store only valid sequences.

Code

```
#include <bits/stdc++.h>
using namespace std;

// Function to check if parentheses string is valid
bool isValid(string s) {
    int balance = 0;
    for (char c : s) {
        if (c == '(') balance++;
        else balance--;
        if (balance < 0) return false;
    }
    return balance == 0;
}

void generateAll(string curr, int n, vector<string>& res) {
    if (curr.length() == 2 * n) {
        if (isValid(curr))
            res.push_back(curr);
        return;
    }
    generateAll(curr + '(', n, res);
    generateAll(curr + ')', n, res);
}

vector<string> generateParenthesis(int n) {
    vector<string> res;
    generateAll("", n, res);
    return res;
}
```

```
int main() {
    int n = 3;
    vector<string> result = generateParenthesis(n);
    for (string s : result)
        cout << s << endl;
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(2^(2n) × n)`
  All possible strings are generated and each is validated.

- **Space Complexity:** `O(n)` recursion depth (excluding result storage).

Optimal Approach (Backtracking)

Idea

Instead of generating invalid strings and filtering them, **build only valid sequences**:

- You can add `'('` if `open < n`.

- You can add `')'` only if `close < open`.

This ensures correctness at every step.

Algorithm

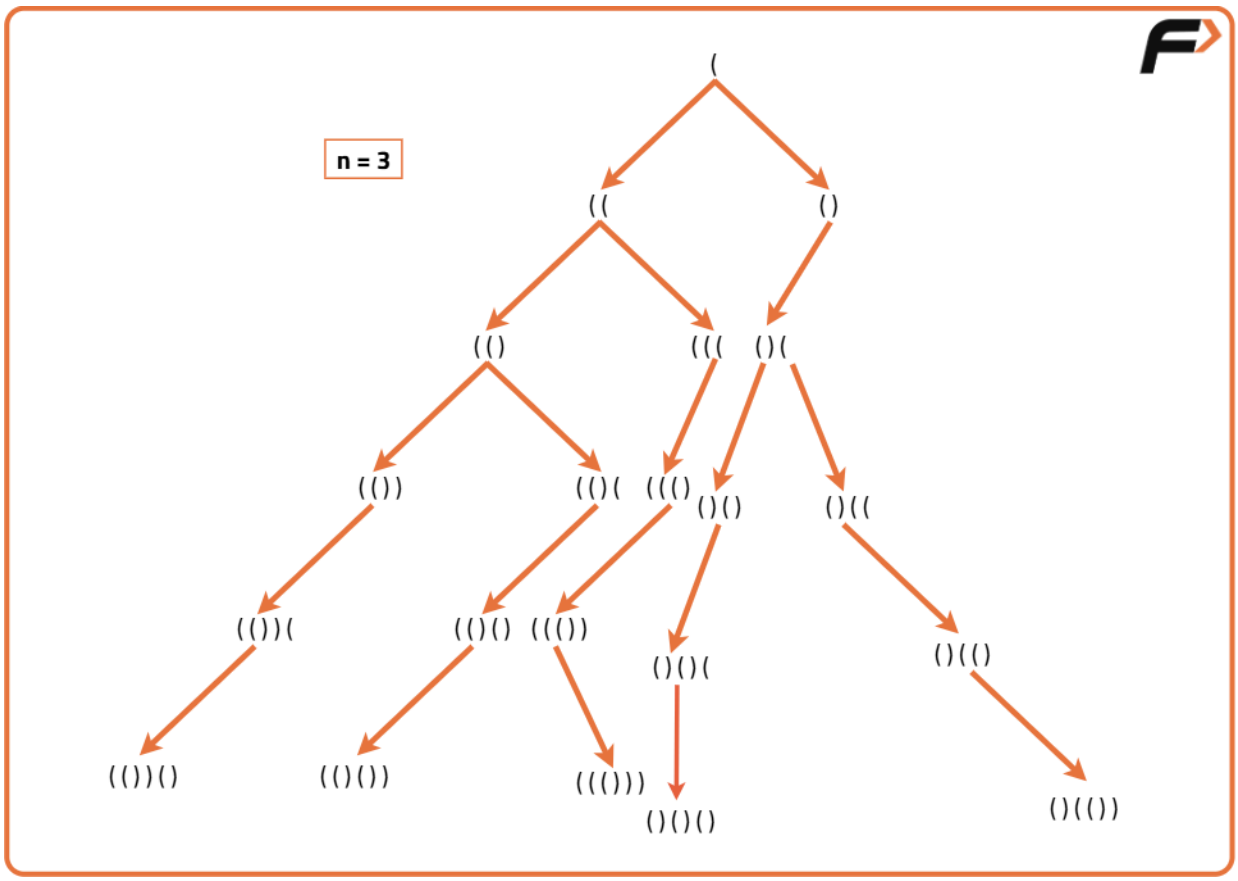
1. Start with an empty string.

2. Maintain counts of `open` and `close`.

3. If `open < n`, add `'('` and recurse.

4. If `close < open`, add `')'` and recurse.

5. When string length becomes 2n, store it.



Code

```
#include <bits/stdc++.h>
using namespace std;

void backtrack(string curr, int open, int close, int n,
vector<string>& res) {
    if (curr.length() == 2 * n) {
        res.push_back(curr);
        return;
    }
    if (open < n)
        backtrack(curr + '(', open + 1, close, n, res);
    if (close < open)
```

```
        backtrack(curr + ')', open, close + 1, n, res);
}

vector<string> generateParenthesis(int n) {
    vector<string> res;
    backtrack("", 0, 0, n, res);
    return res;
}

int main() {
    int n = 3;
    vector<string> result = generateParenthesis(n);
    for (string s : result)
        cout << s << endl;
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(C(n) \times n)$
  where $C(n)$ is the nth Catalan number (number of valid sequences).

- **Space Complexity:** $O(n)$ recursion depth
  plus space to store all valid combinations.

8. Power Set: Print all the possible subsequences of a String

Explanation

You are given a string `str`.
A **subsequence** is formed by choosing **some characters** from the string **without changing**

their relative order.
You can either **include or exclude** each character, so for a string of length n, there are `2^n` possible subsequences (including the empty one).

Empty subsequence is usually generated internally, but in many outputs it is skipped.

Example
Input: `str = "abc"`
Possible subsequences:

```
a, ab, abc, ac, b, bc, c
```

Approach 1: Bit Manipulation

Explanation
Each character has two choices:

- `0` → exclude
- 1 → include

So every subsequence can be represented using a binary number of length n.

For `"abc"` (n = 3):

```
001 → c
010 → b
011 → bc
101 → ac
```

Algorithm

1. Let n be the length of the string.
2. Total subsequences = `2^n`.
3. Loop from `0` to `2^n - 1`.

4. For each number, check its bits:
    - If the i-th bit is set, include `s[i]`.
5. Store the generated subsequence.

str = "abc" **length = 3**

∴ total subsequences = $2^n = 2^3 = 8$

Decimal	Binary	Chosen Character	Subsequence
0	000	none	""
1	001	'c'	"c"
2	010	'b'	"b"
3	011	'b', 'c'	"bc"
4	100	'a'	"a"
5	101	'a', 'c'	"ac"
6	110	'a', 'b'	"ab"
7	111	'a', 'b', 'c'	"abc"

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<string> getSubsequences(string s) {
        int n = s.size();
        int total = 1 << n;
        vector<string> ans;

        for (int mask = 1; mask < total; mask++) {
            string curr = "";
            for (int i = 0; i < n; i++) {
                if (mask & (1 << i)) {
                        // If i-th bit of mask is set, include s[i]
                    curr.push_back(s[i]);
                }
            }
```

```
            ans.push_back(curr);
        }
        return ans;
    }
};

int main() {
    string s = "abc";
    Solution sol;

    vector<string> res = sol.getSubsequences(s);
    for (string x : res) {
        cout << x << endl;
    }
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(n * 2^n)`
- **Space Complexity:** `O(n * 2^n)` (to store all subsequences)

Approach 2: Recursion (Include / Exclude)

Explanation
For every character, you have two choices:

- Exclude it
- Include it

This forms a binary recursion tree.

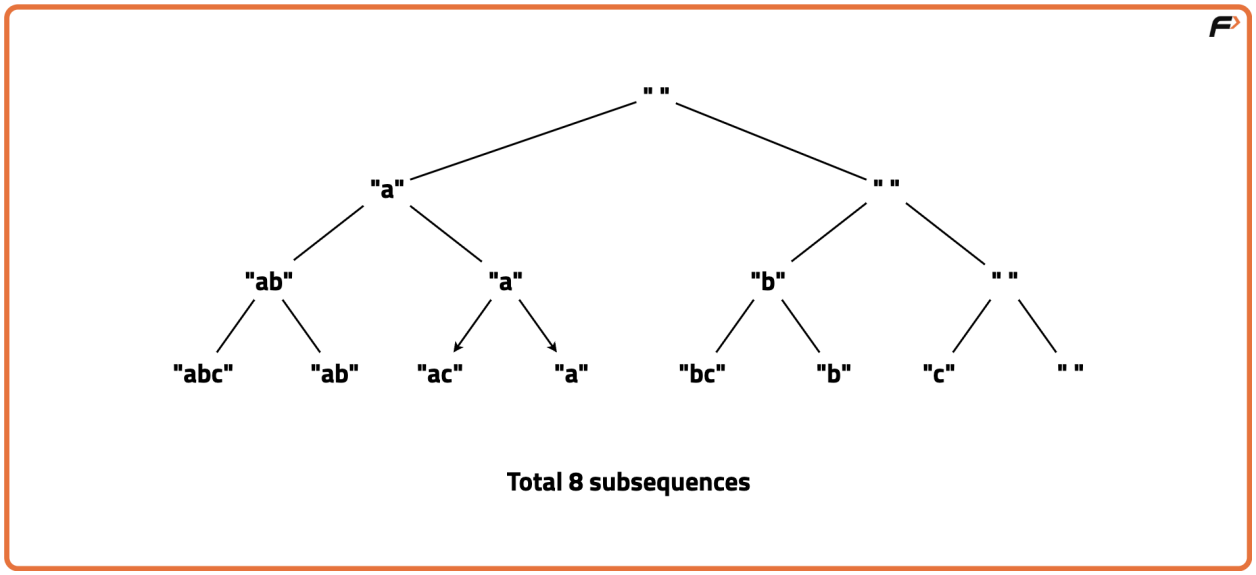
At each index:

- Move forward **without** including the character.
- Move forward **after including** the character.

When you reach the end of the string, the current string is a valid subsequence.

Algorithm

1. Start from index `0` with an empty string.
2. At each index:
    - Recurse without taking the character.
    - Recurse after taking the character.
3. When index reaches n, store the current subsequence.



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
```

```
public:
    void helper(string &s, int idx, string &curr, vector<string> &ans)
{
        if (idx == s.size()) {
            if (!curr.empty())
                ans.push_back(curr);
            return;
        }

        // Exclude current character
        helper(s, idx + 1, curr, ans);

        // Include current character
        curr.push_back(s[idx]);
        helper(s, idx + 1, curr, ans);
        curr.pop_back();
    }

    vector<string> getSubsequences(string s) {
        vector<string> ans;
        string curr = "";
        helper(s, 0, curr, ans);
        return ans;
    }
};

int main() {
    string s = "abc";
    Solution sol;

    vector<string> res = sol.getSubsequences(s);
    for (string x : res) {
        cout << x << endl;
    }
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(n * 2^n)`
- **Space Complexity:** `O(n)` recursion stack
  (`O(n * 2^n)` including result storage)

Learn All Patterns of Subsequences (Theory)

9. What are Subsequences?

A **subsequence** is a sequence derived from another sequence by deleting some or no elements without changing the order of the remaining elements. Unlike substrings, the elements of a subsequence are not required to occupy consecutive positions in the original sequence.

For example, in the string `"abc"`, the subsequences include: `"a"`, `"b"`, `"c"`, `"ab"`, `"ac"`, `"bc"`, and `"abc"`.

Total Subsequences of a String

For a string of length `n`, the total number of subsequences (excluding the empty one) is **$2^n - 1$**.

This is because each character has 2 options: either be included or not. Thus, 2 × 2 × ... (n times) = 2^n. We subtract 1 to exclude the empty subsequence if not required.

Subsequence vs Substring

Subsequence	Substring
Can skip characters, but order matters	Must be contiguous

Number of subsequences is 2^n	Number of substrings is $n(n+1)/2$
"ace" is a subsequence of "abcde"	"abc" is a substring of "abcde"

Patterns and Problems Based on Subsequences

- Generate All Subsequences – Basic recursion or backtracking
- Count Subsequences with Specific Property – e.g. sum = target
- Longest Increasing Subsequence (LIS) – Classic DP problem
- Subsequences with K elements – Use recursion with element count
- Print all subsequences with sum = K – Variation of subset sum

Recursive Structure for Generating Subsequences

We use the idea of "pick or not pick" for each character in the string or element in the array.

```
function recurse(index, current):
    if index == n:
        print current
        return
    // Pick the current element
    recurse(index + 1, current + arr[index])
    // Do not pick the current element
    recurse(index + 1, current)
```

Use Cases in Problem Solving

- Subset sum problems
- Count/print subsequences with a given sum
- Dynamic programming on subsequences (e.g. LIS, LCS, Count Palindromic Subsequences)
- Bitmasking based optimizations

Time & Space Complexity

Operation	Time Complexity	Space Complexity
Generate All Subsequences	$O(2^n)$	O(n)
Check for specific sum	$O(2^n)$	O(n)
LIS (DP)	$O(n^2)$ or O(n log n)	O(n)

Note: Generating all subsequences has exponential complexity. Use it wisely for small `n` (usually ≤ 20).

Conclusion

Understanding subsequences and the ability to manipulate them through recursion and dynamic programming unlocks solutions to a wide variety of problems. From classic LIS and subset-sum to interview questions on palindromic subsequences, the concept is foundational in problem-solving.

10. Count all subsequences with sum K

Explanation

You are given an integer array `nums` and an integer `k`.

Your task is to count **all non-empty subsequences** whose **sum of elements is exactly equal to k**.

A **subsequence** means:

- You can choose or skip any element
- Order must remain the same
- Elements do **not** have to be contiguous

So for every element, you always have **two choices**:

1. Include it in the subsequence
2. Exclude it from the subsequence

This naturally leads to a **recursive include / exclude approach**.

Example 1

Input:

```
nums = [4, 9, 2, 5, 1], k = 10
```

Valid subsequences:

```
[9, 1]
[4, 5, 1]
```

Output:

2

Example 2

Input:

```
nums = [4, 2, 10, 5, 1, 3], k = 5
```

Valid subsequences:

```
[4, 1]
[2, 3]
[5]
```

Output:

3

Recursive Approach (Include / Exclude)

Explanation of the idea
We process the array from left to right.

At every index:

- Either **include** the current element in the sum
- Or **exclude** it and move forward

We stop recursion when:

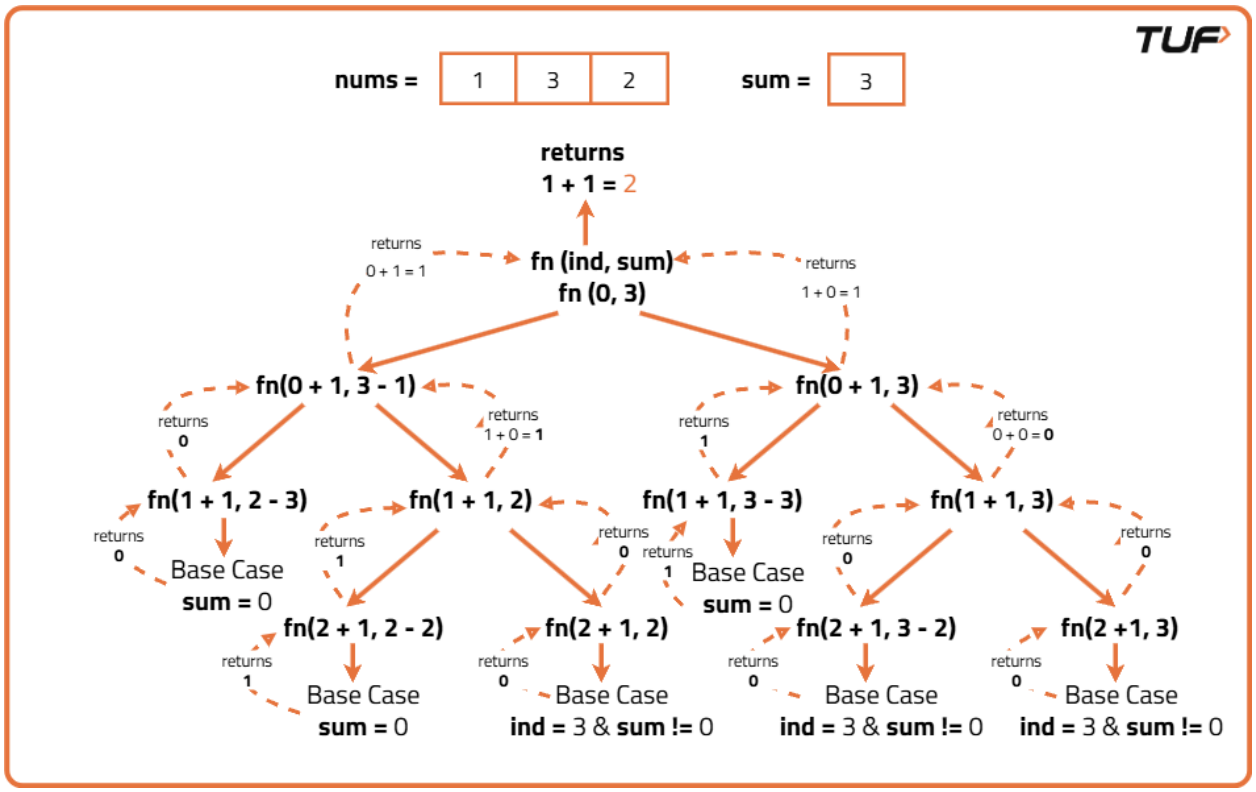
- The sum becomes `0` → one valid subsequence found
- The sum becomes negative → invalid path
- We reach the end of the array → no valid subsequence

The total answer is the **sum of both recursive choices**.

Algorithm

1. Start recursion from index `0` with target sum k

2. At each index:
    - Include the current element → reduce sum
    - Exclude the current element → keep sum same
3. If sum becomes `0`, return 1
4. If sum < 0 or index reaches array size, return `0`
5. Add results of both choices



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
```

```
private:
    int solve(int index, int sum, vector<int>& nums) {
        // If exact sum achieved
        if (sum == 0) return 1;

        // If sum becomes negative or index out of range
        if (sum < 0 || index == nums.size()) return 0;

        // Include current element
        int take = solve(index + 1, sum - nums[index], nums);

        // Exclude current element
        int notTake = solve(index + 1, sum, nums);

        return take + notTake;
    }

public:
    int countSubsequenceWithTargetSum(vector<int>& nums, int k) {
        return solve(0, k, nums);
    }
};

int main() {
    Solution sol;
    vector<int> nums = {4, 9, 2, 5, 1};
    int k = 10;

    cout << sol.countSubsequenceWithTargetSum(nums, k) << endl;
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(2^n)`
  Every element has two choices (include / exclude)

- **Space Complexity:** `O(n)`
  Due to recursion stack depth

11. Check if there exists a subsequence with sum K

Explanation
You are given an integer array `nums` and an integer `k`.
You need to check **whether at least one non-empty subsequence exists whose sum is exactly k**.

A **subsequence** means:

- You may choose or skip any element
- Order remains the same
- Elements do not need to be contiguous

For every element, you always have **two choices**:

- include it in the sum
- exclude it from the sum

If **any path** leads to sum `k`, return `true`, otherwise return `false`.

Example 1

Input:

```
nums = [1, 2, 3, 4, 5], k = 8
```

Valid subsequences:

```
[1, 2, 5]
[1, 3, 4]
[3, 5]
```

Output:

```
true
```

Example 2

Input:

```
nums = [4, 3, 9, 2], k = 10
```

No subsequence forms sum 10
Output:

```
false
```

Recursive Include / Exclude Approach

Idea
We process the array index by index.

At each index:

- include the current element → reduce sum
- exclude the current element → sum remains same

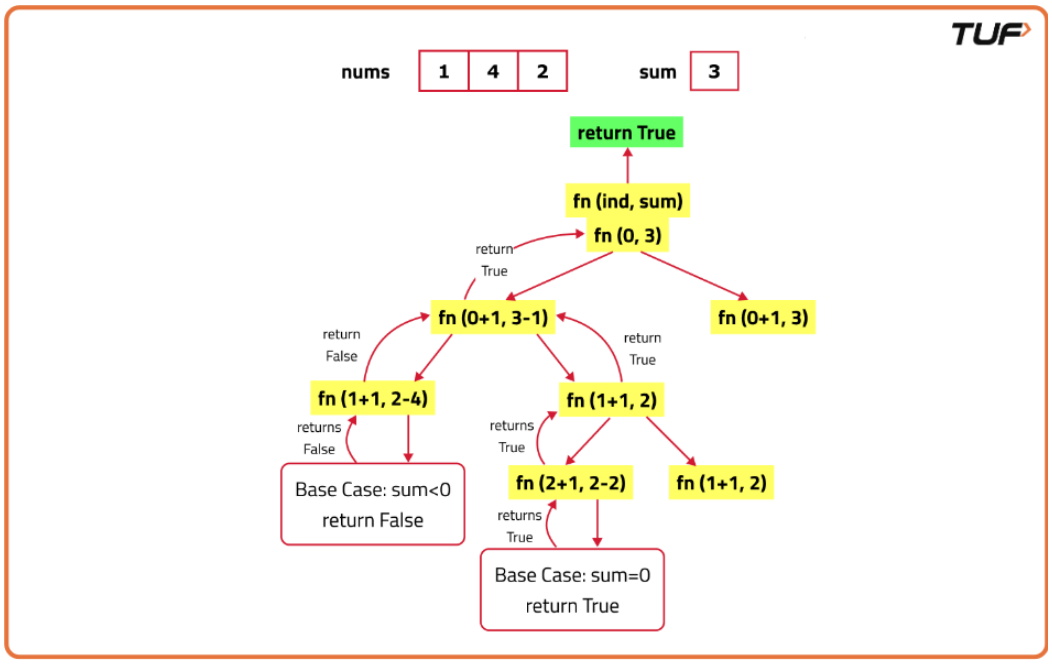
We stop recursion when:

- sum becomes `0` → subsequence found

- index reaches end → no valid subsequence on this path

Algorithm

1. Start recursion from index `0` with target sum k
2. At each index:
    - include current element
    - exclude current element
3. If sum becomes `0`, return `true`
4. If array ends, return `false`
5. Return `true` if **any path** returns `true`



Code

```
#include <bits/stdc++.h>
using namespace std;
```

```
class Solution {
private:
    bool solve(int index, int sum, vector<int>& nums) {
        // If sum becomes 0, subsequence exists
        if (sum == 0) return true;

        // If index is out of bounds
        if (index == nums.size()) return false;

        // Include current element OR exclude it
        return solve(index + 1, sum - nums[index], nums) ||
               solve(index + 1, sum, nums);
    }

public:
    bool checkSubsequenceSum(vector<int>& nums, int k) {
        return solve(0, k, nums);
    }
};

int main() {
    Solution sol;
    vector<int> nums = {1, 2, 3, 4, 5};
    int k = 8;

    cout << sol.checkSubsequenceSum(nums, k);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(2^n)`
  Each element has two choices (include / exclude)

- **Space Complexity:** `O(n)`
  Due to recursion stack depth

12. Combination Sum – I

You are given an array of **distinct integers** and a **target** value.
Your task is to find **all unique combinations** of numbers such that the **sum equals the target**.

Rules:

- You can **use the same number unlimited times**
- Order of numbers inside a combination does not matter
- Each combination must be unique based on frequency of elements

This is a classic **recursion + backtracking** problem.

Example 1

Input:

```
array = [2, 3, 6, 7], target = 7
```

Valid combinations:

```
[2, 2, 3]
[7]
```

Example 2

Input:

```
array = [2], target = 1
```

Output:

```
[]
```

Recursive Pick / Not-Pick Approach

Idea
At every index, we decide:

- **Pick** the current element (stay on same index because reuse is allowed)
- **Not pick** the current element (move to next index)

We keep track of:

- current index
- remaining target
- current combination (ds)

When:

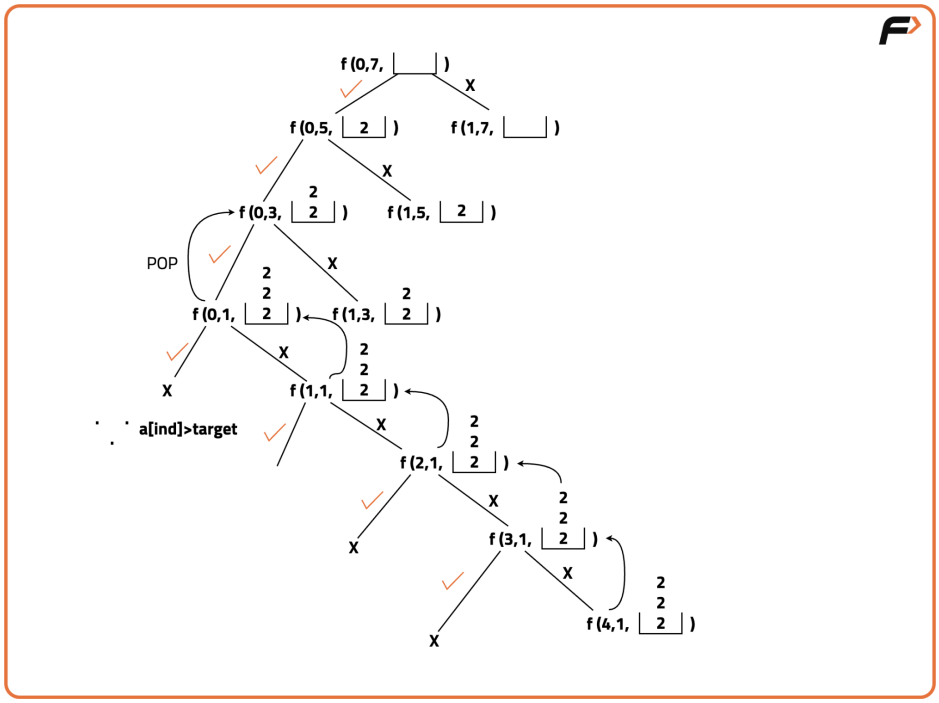
- `target == 0` → valid combination found
- `index == array.size()` → stop recursion

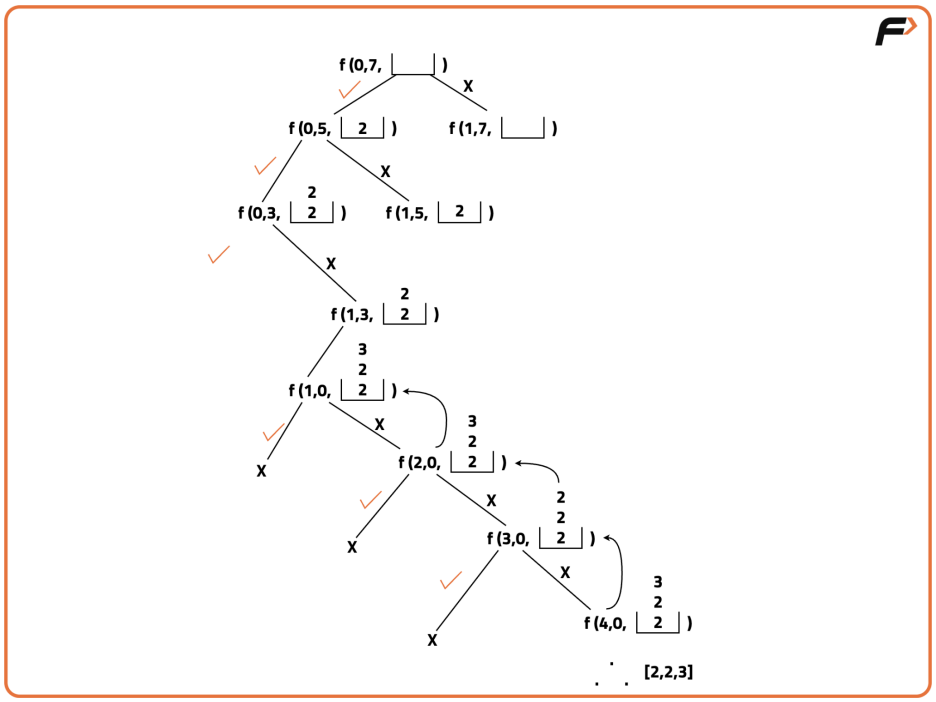
Algorithm

1. Start recursion from index `0`
2. If current element ≤ target:
    - pick it
    - reduce target
    - stay on same index
3. Always explore the not-pick case:

- move to next index

4. Backtrack after each recursive call
5. Collect all valid combinations





Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
private:
    void findCombination(int index, int target, vector<int>& arr,
                         vector<vector<int>>& ans, vector<int>& ds) {

        // If we have processed all elements
        if (index == arr.size()) {
            if (target == 0) {
                ans.push_back(ds);
            }
            return;
        }

        // Pick the current element if possible
        if (arr[index] <= target) {
            ds.push_back(arr[index]);
            findCombination(index, target - arr[index], arr, ans, ds);
            ds.pop_back(); // backtrack
        }

        // Do not pick the current element
        findCombination(index + 1, target, arr, ans, ds);
    }

public:
    vector<vector<int>> combinationSum(vector<int>& candidates, int
target) {
        vector<vector<int>> ans;
        vector<int> ds;
        findCombination(0, target, candidates, ans, ds);
        return ans;
    }
```

```
};

int main() {
    Solution obj;
    vector<int> candidates = {2, 3, 6, 7};
    int target = 7;

    vector<vector<int>> result = obj.combinationSum(candidates,
target);

    for (auto &comb : result) {
        for (int x : comb) cout << x << " ";
        cout << endl;
    }
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** Exponential (depends on number of combinations formed)
- **Space Complexity:**
    - $O(k \times x)$ to store results
    - $O(k)$ recursion depth
      (k = average length of a combination, x = number of combinations)

This approach strictly follows **pick / not-pick recursion**, allows reuse of elements, and avoids duplicate combinations.

13. Combination Sum II – Find All Unique Combinations

Explanation

You are given an array of integers `candidates` and a target value `target`.
Your task is to find **all unique combinations** where the chosen numbers sum to `target`.

Rules:

- **Each number can be used at most once** in a combination
- The input array **may contain duplicates**
- The result must not contain duplicate combinations

This problem is very similar to **Combination Sum I**, but with **two key differences**:

1. An element can be picked **only once**
2. Duplicate combinations must be **avoided**

Example 1

Input:

```
candidates = [10,1,2,7,6,1,5], target = 8
```

Output:

```
[ [1,1,6], [1,2,5], [1,7], [2,6] ]
```

Example 2

Input:

```
candidates = [2,5,2,1,2], target = 5
```

Output:

```
[ [1,2,2], [5] ]
```

Core Idea (Backtracking with Duplicate Handling)

We still use **recursion + backtracking**, but with these rules:

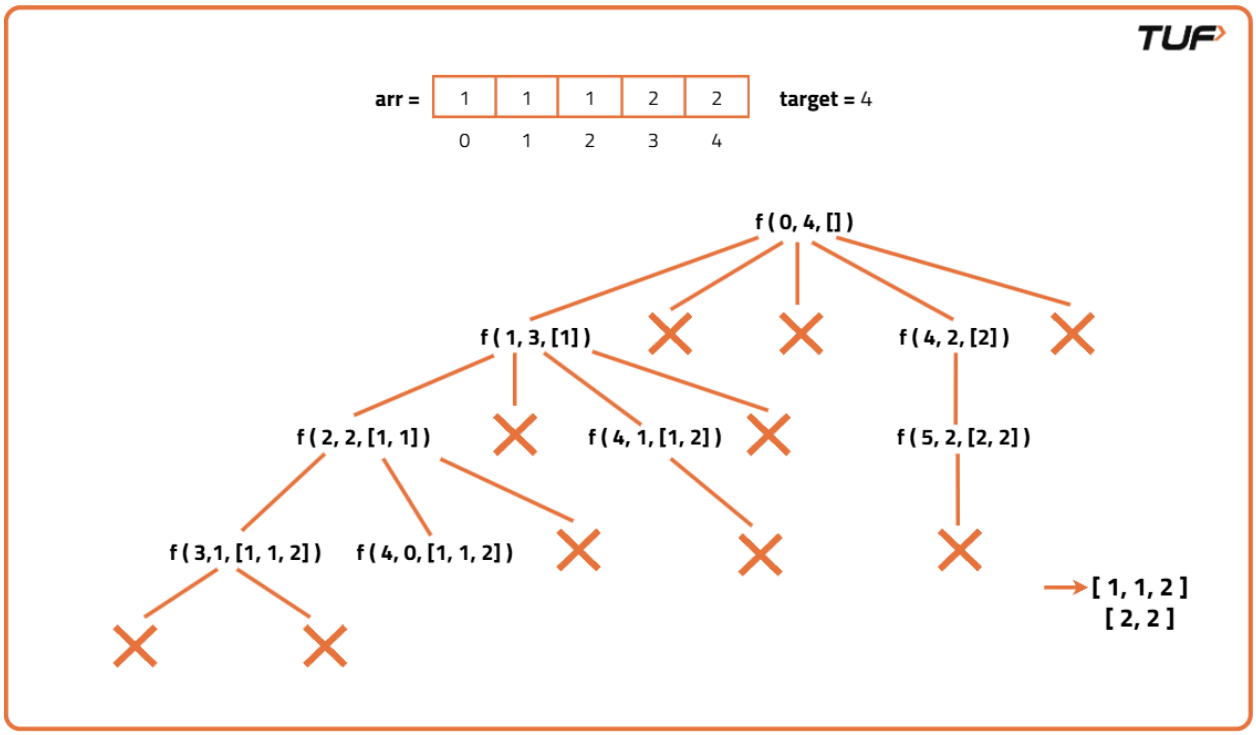
1. **Sort the array first**
    - Helps in skipping duplicates
    - Helps in early stopping when element > target
2. Use a loop instead of pick / not-pick recursion
    - At every recursive level, try all possible elements starting from `index`
3. **Skip duplicates**
    - If `arr[i] == arr[i-1]` and `i > index`, skip it
4. Move to **next index (`i + 1`)** after picking an element
    - Ensures each number is used only once

Algorithm

1. Sort the candidates array
2. Start recursion from index `0`
3. For every index `i`:

- Skip duplicate values
- Stop if `arr[i] > target`
- Pick `arr[i]` and recurse with:
  - `target - arr[i]`
  - `i + 1`

4. If `target == 0`, store the combination
5. Backtrack after recursion



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
```

```
private:
    void findCombination(int index, int target, vector<int>& arr,
                         vector<vector<int>>& ans, vector<int>& ds) {

        // If target becomes 0, valid combination found
        if (target == 0) {
            ans.push_back(ds);
            return;
        }

        for (int i = index; i < arr.size(); i++) {

            // Skip duplicates
            if (i > index && arr[i] == arr[i - 1]) continue;

            // Early stopping
            if (arr[i] > target) break;

            // Pick current element
            ds.push_back(arr[i]);

            // Move to next index (each element used once)
            findCombination(i + 1, target - arr[i], arr, ans, ds);

            // Backtrack
            ds.pop_back();
        }
    }

public:
    vector<vector<int>> combinationSum2(vector<int>& candidates, int
target) {
        sort(candidates.begin(), candidates.end());
        vector<vector<int>> ans;
        vector<int> ds;
        findCombination(0, target, candidates, ans, ds);
        return ans;
    }
```

```
};

int main() {
    Solution sol;
    vector<int> candidates = {10, 1, 2, 7, 6, 1, 5};
    int target = 8;

    vector<vector<int>> result = sol.combinationSum2(candidates,
target);

    for (auto &comb : result) {
        for (int x : comb) cout << x << " ";
        cout << endl;
    }
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(2^n \times k)$
  Each element can be chosen or skipped, and storing a combination takes $O(k)$

- **Space Complexity:**

  - $O(k \times x)$ to store all valid combinations

  - $O(k)$ recursion stack
    (k = average length of combination, x = number of valid combinations)

14. Subset Sum – Sum of All Subsets

Explanation
You are given an array of integers.
Your task is to **generate the sum of every possible subset** of this array and **print all those sums in increasing order**.

A subset can be:

- empty
- contain some elements
- contain all elements

For an array of size `N`, total subsets = `2^N`, so we will also get `2^N` subset sums.

Example 1

Input:

```
arr = {5, 2, 1}
```

All subsets:

```
[] → 0
[1] → 1
[2] → 2
[2,1] → 3
[5] → 5
[5,1] → 6
[5,2] → 7
[5,2,1] → 8
```

Output:

```
0 1 2 3 5 6 7 8
```

Example 2

Input:

```
arr = {3, 1, 2}
```

Output:

```
0 1 2 3 3 4 5 6
```

Approach 1: Bitmasking

Idea

Each subset can be represented using a binary number of length `N`.

- `0` → element excluded
- `1` → element included

We iterate from `0` to `(2^N - 1)` and for each number, calculate the subset sum based on set bits.

Algorithm

1. Let `n` be the size of the array
2. Iterate `mask` from `0` to `(1 << n) - 1`
3. For each mask:
    - Initialize `sum = 0`
    - For each bit `i` from `0` to `n-1`

- If `i`th bit is set, add `arr[i]` to `sum`

4. Store `sum` in result list
5. Sort the result list
6. Return / print it

Code (Bitmasking)

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<int> subsetSums(vector<int>& arr) {
        int n = arr.size();
        vector<int> sums;

        for (int mask = 0; mask < (1 << n); mask++) {
            int sum = 0;
            for (int i = 0; i < n; i++) {
                if (mask & (1 << i)) {
                    sum += arr[i];
                }
            }
            sums.push_back(sum);
        }

        sort(sums.begin(), sums.end());
        return sums;
    }
};

int main() {
    Solution sol;
    vector<int> arr = {5, 2, 1};
```

```
    vector<int> result = sol.subsetSums(arr);

    for (int x : result) cout << x << " ";
    return 0;
}
```

Complexity (Bitmasking)

- **Time Complexity:** `O(2^N * N)`
- **Space Complexity:** `O(2^N)`

Approach 2: Recursion (Pick / Not Pick)

Idea
At every index, you have **two choices**:

- include the element
- exclude the element

We carry a running sum and once we reach the end of the array, we store that sum.

Algorithm

1. Create a recursive function with parameters:
    - `index`
    - `currentSum`
2. If `index == N`:

- store `currentSum`
- return

3. Recursive calls:
   - include current element → `currentSum + arr[index]`
   - exclude current element → `currentSum`
4. Start recursion from index `0` and sum `0`
5. Sort the result list

Code (Recursive)

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    void findSums(int index, int currentSum,
                  vector<int>& arr, vector<int>& sums) {

        if (index == arr.size()) {
            sums.push_back(currentSum);
            return;
        }

        // Include current element
        findSums(index + 1, currentSum + arr[index], arr, sums);

        // Exclude current element
        findSums(index + 1, currentSum, arr, sums);
    }

    vector<int> subsetSums(vector<int>& arr) {
        vector<int> sums;
```

```
        findSums(0, 0, arr, sums);
        sort(sums.begin(), sums.end());
        return sums;
    }
};

int main() {
    Solution sol;
    vector<int> arr = {5, 2, 1};
    vector<int> result = sol.subsetSums(arr);

    for (int x : result) cout << x << " ";
    return 0;
}
```

Complexity (Recursive)

- **Time Complexity:** `O(2^N)`
- **Space Complexity:**
    - `O(2^N)` for storing sums
    - `O(N)` recursion stack

15. Subset II – Print All Unique Subsets

Explanation

You are given an integer array `nums` that **may contain duplicate elements**.

Your task is to generate the **power set (all possible subsets)** such that **no duplicate subset appears in the final answer**.

A subset:

- can be empty
- can contain some or all elements
- order inside a subset does not matter

Because duplicates exist in the input array, care must be taken to **avoid printing the same subset more than once**.

Example 1

Input:

```
nums = [1, 2, 2]
```

Output:

```
[ [], [1], [1,2], [1,2,2], [2], [2,2] ]
```

Explanation:
Although `[1,2]` can be formed in multiple ways, it is printed **only once**.

Example 2

Input:

```
nums = [1]
```

Output:

```
[ [], [1] ]
```

Brute Force Approach

Idea
Generate **all possible subsets** using recursion (pick / not pick).
Since duplicates will appear, store all subsets inside a **set**, which automatically removes duplicates.

Algorithm

1. Sort the array (for consistent subset formation)
2. Use recursion to generate all subsets:
    - pick the current element
    - do not pick the current element
3. When the index reaches the end:
    - insert the current subset into a `set`
4. Convert the set into a vector and return it

Code (Brute Force)

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    void findSubsets(int ind, vector<int>& nums,
                     vector<int>& ds, set<vector<int>>& st) {
        if (ind == nums.size()) {
            st.insert(ds);
            return;
```

```
        }

        // Pick
        ds.push_back(nums[ind]);
        findSubsets(ind + 1, nums, ds, st);
        ds.pop_back();

        // Not pick
        findSubsets(ind + 1, nums, ds, st);
    }

    vector<vector<int>> subsetsWithDup(vector<int>& nums) {
        sort(nums.begin(), nums.end());
        set<vector<int>> st;
        vector<int> ds;

        findSubsets(0, nums, ds, st);

        return vector<vector<int>>(st.begin(), st.end());
    }
};

int main() {
    Solution sol;
    vector<int> nums = {1, 2, 2};
    vector<vector<int>> ans = sol.subsetsWithDup(nums);

    for (auto &subset : ans) {
        cout << "[ ";
        for (int x : subset) cout << x << " ";
        cout << "] ";
    }
    return 0;
}
```

Complexity (Brute Force)

- **Time Complexity:** `O(N² * 2^N)`
- **Space Complexity:** `O(N * 2^N)`

Reason:
All subsets are generated and stored in a set, which adds extra overhead.

Optimal Approach (Backtracking with Skipping Duplicates)

Idea
Instead of generating duplicates and removing them later, **avoid creating duplicates in the first place**.

Key observation:

- After sorting, **duplicate elements will be adjacent**
- While looping, if the current element is the same as the previous one and we are at the same recursion level, **skip it**

Algorithm

1. Sort the array
2. Use backtracking:
    - Add current subset to result
    - Loop from `start` index to end
    - If `i > start` and `nums[i] == nums[i-1]`, skip
    - Pick element → recurse → backtrack

3. Return the result

Code (Optimal)

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    void backtrack(int start, vector<int>& nums,
                   vector<int>& current, vector<vector<int>>& result)
{

        result.push_back(current);

        for (int i = start; i < nums.size(); i++) {
            if (i > start && nums[i] == nums[i - 1]) continue;

            current.push_back(nums[i]);
            backtrack(i + 1, nums, current, result);
            current.pop_back();
        }
    }

    vector<vector<int>> subsetsWithDup(vector<int>& nums) {
        sort(nums.begin(), nums.end());
        vector<vector<int>> result;
        vector<int> current;

        backtrack(0, nums, current, result);
        return result;
    }
};

int main() {
    Solution sol;
    vector<int> nums = {1, 2, 2};
```

```
    vector<vector<int>> ans = sol.subsetsWithDup(nums);

    for (auto &subset : ans) {
        cout << "[";
        for (int i = 0; i < subset.size(); i++) {
            cout << subset[i];
            if (i + 1 < subset.size()) cout << ",";
        }
        cout << "] ";
    }
    return 0;
}
```

Complexity (Optimal)

- **Time Complexity:** `O(2^N)`
- **Space Complexity:**
    - `O(N)` recursion stack
    - Output storage: `O(2^N)`

16. Combination Sum III

Explanation

You are given two integers k and n.

You need to find **all unique combinations of exactly k numbers** such that:

- Numbers are chosen only from **1 to 9**

- Each number is used **at most once**
- The sum of chosen numbers is exactly **n**
- No duplicate combinations are allowed

Order of combinations does not matter.

Example 1

Input:

`k = 3, n = 7`

Output:

`[[1, 2, 4]]`

Explanation:
Only `1 + 2 + 4 = 7` using exactly 3 numbers.

Example 2

Input:

`k = 3, n = 9`

Output:

`[[1,2,6], [1,3,5], [2,3,4]]`

Approach

Idea
This is a classic backtracking problem where we try to build combinations step by step.

At every step:

- Pick a number from 1 to 9
- Use each number only once
- Keep track of remaining sum
- Stop early if:
    - sum becomes negative
    - combination size exceeds k

Algorithm

1. Start from number 1
2. Maintain:
    - remaining sum
    - current combination
3. If remaining sum is `0` and size of combination is k, store it
4. If sum < 0 or size > k, stop recursion
5. Try next numbers from `last + 1` to 9
6. Backtrack after each recursive call

Code

```
#include <bits/stdc++.h>
```

```cpp
using namespace std;

class Solution {
private:
    void solve(int sum, int start, int k,
               vector<int>& curr,
               vector<vector<int>>& ans) {

        if (sum == 0 && curr.size() == k) {
            ans.push_back(curr);
            return;
        }

        if (sum < 0 || curr.size() > k) return;

        for (int i = start; i <= 9; i++) {
            curr.push_back(i);
            solve(sum - i, i + 1, k, curr, ans);
            curr.pop_back();
        }
    }

public:
    vector<vector<int>> combinationSum3(int k, int n) {
        vector<vector<int>> ans;
        vector<int> curr;
        solve(n, 1, k, curr, ans);
        return ans;
    }
};

int main() {
    Solution sol;
    int k = 3, n = 9;
    vector<vector<int>> res = sol.combinationSum3(k, n);

    for (auto &v : res) {
        for (int x : v) cout << x << " ";
```

```
        cout << endl;
    }
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(2^9 * k)`
  We explore all subsets of numbers from 1 to 9.

- **Space Complexity:** `O(k)`
  Used by recursion stack and current combination.

17. Letter Combinations of a Phone Number

Explanation
You are given a string `digits` containing characters from `'2'` to `'9'`.
Each digit maps to certain letters on a phone keypad (like old mobile phones).
Your task is to generate **all possible strings** by replacing each digit with one of its mapped letters, keeping the order same.

If the input string has length n, then each position contributes multiple choices, and we generate combinations by choosing one letter per digit.

Example 1

Input:

```
digits = "34"
```

Explanation:

- `3 → "def"`
- `4 → "ghi"`

All combinations formed by picking one letter from `"def"` and one from `"ghi"`:

`["dg","dh","di","eg","eh","ei","fg","fh","fi"]`

Example 2

Input:

```
digits = "3"
```

Output:

`["d","e","f"]`

Approach

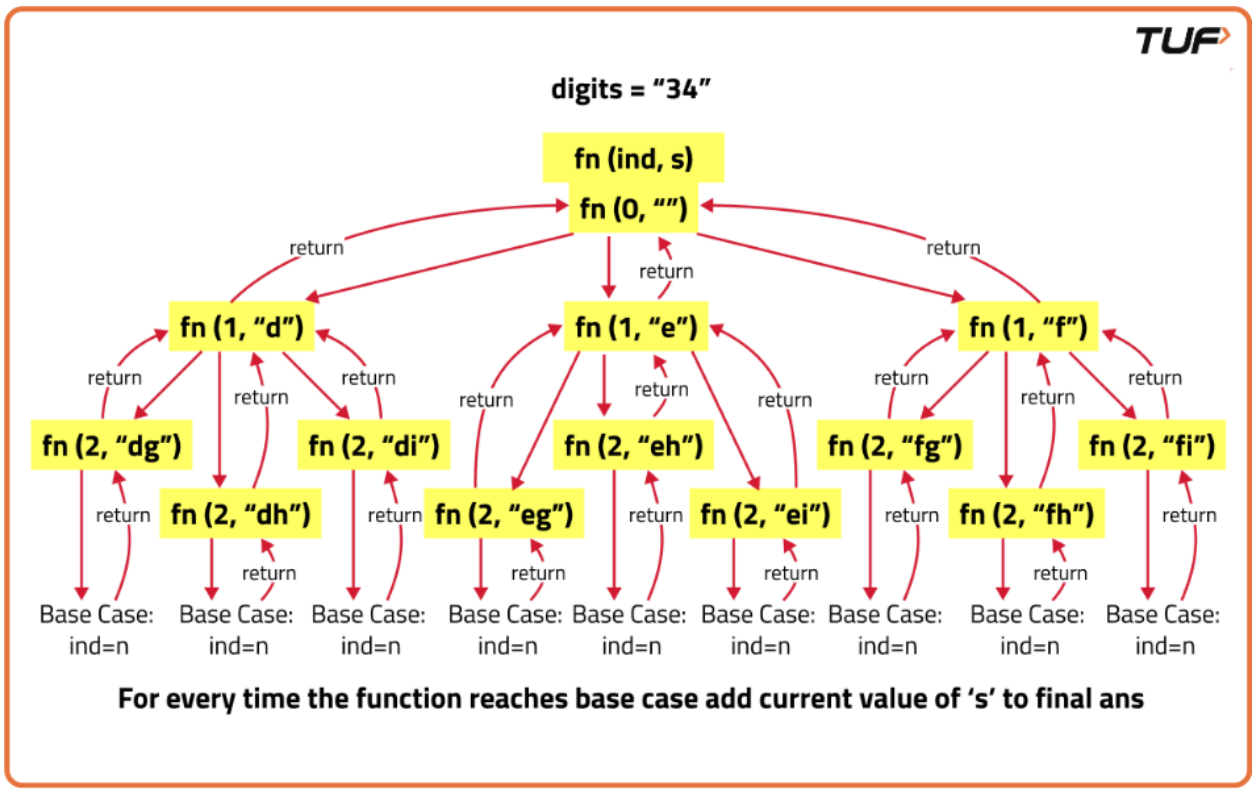
Idea
This is a backtracking problem.
At each index, we:

- Look at the digit
- Try all characters mapped to that digit
- Move to the next index

When we reach the end of the digit string, the current built string is a valid combination.

Algorithm

1. Create a mapping from digits (2−9) to letters.
2. Use a recursive function with:
    - current index
    - current string being formed
3. If index equals length of digits, store the string.
4. Otherwise:
    - For the current digit, loop through all mapped letters
    - Append one letter and recurse to next index



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
private:
    void solve(int index, string &digits, string &curr,
               vector<string> &ans, string mp[]) {

        if (index == digits.size()) {
            ans.push_back(curr);
            return;
        }

        int digit = digits[index] - '0';
```

```
        for (char ch : mp[digit]) {
            curr.push_back(ch);
            solve(index + 1, digits, curr, ans, mp);
            curr.pop_back();
        }
    }

public:
    vector<string> letterCombinations(string digits) {
        if (digits.size() == 0) return {};

        string mp[] = {
            "", "", "abc", "def", "ghi",
            "jkl", "mno", "pqrs", "tuv", "wxyz"
        };

        vector<string> ans;
        string curr = "";
        solve(0, digits, curr, ans, mp);
        return ans;
    }
};

int main() {
    Solution sol;
    string digits = "34";
    vector<string> res = sol.letterCombinations(digits);

    for (auto &s : res) cout << s << " ";
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(4^n)`
  Each digit can contribute up to 4 letters.

- **Space Complexity:** `O(n)`
  Recursion stack depth and temporary string length.

18. Palindrome Partitioning

Explanation
You are given a string `s`. Your task is to split (partition) the string into substrings such that **every substring is a palindrome**.
A palindrome reads the same from left to right and right to left.

You must return **all possible such partitions**.

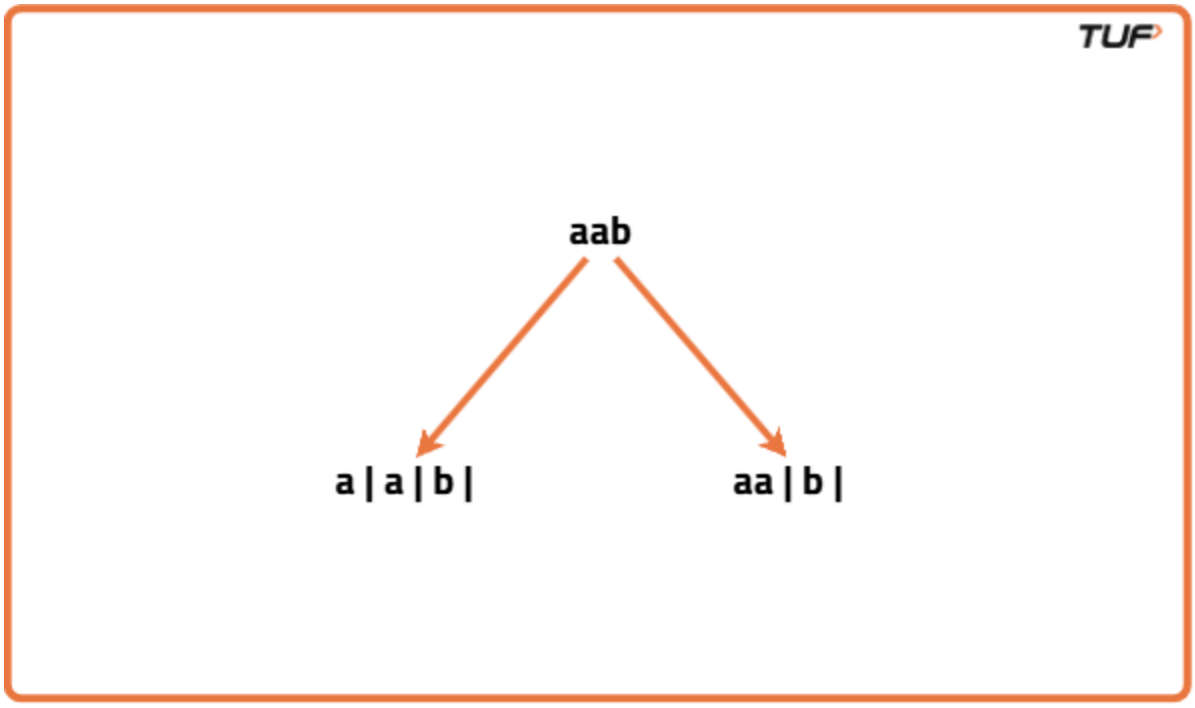
Example 1

Input:

```
s = "aab"
```

Output:

```
["a","a","b"]
["aa","b"]
```



Explanation:

- `"a" | "a" | "b"` → all are palindromes
- `"aa" | "b"` → both are palindromes

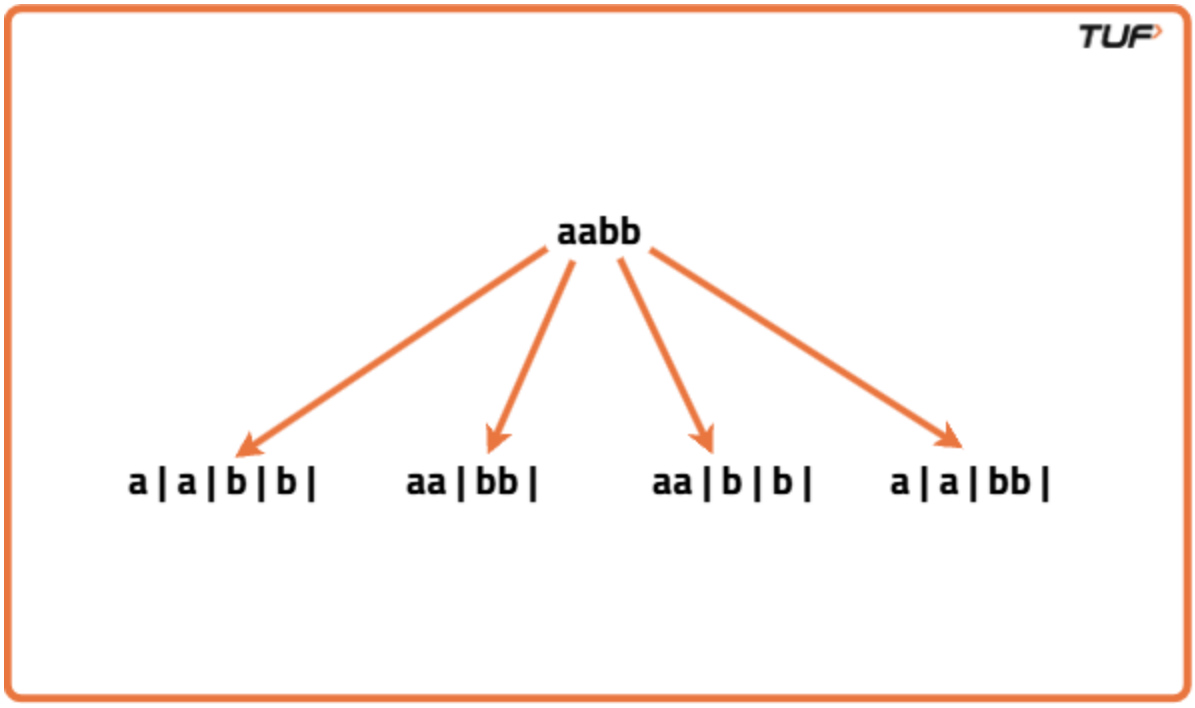
Example 2

Input:

```
s = "aabb"
```

Output:

```
["a","a","b","b"]
["a","a","bb"]
["aa","b","b"]
["aa","bb"]
```



Approach

We use **recursion + backtracking**.

At every index, we try to cut the string into substrings.
For each substring:

- If it is a palindrome → choose it
- Recurse for the remaining string
- Backtrack to try other possibilities

The recursion ends when the full string is consumed.

Code

```
#include <bits/stdc++.h>
using namespace std;
```

```
class Solution {
public:
    bool isPalindrome(string &s, int l, int r) {
        while (l < r) {
            if (s[l] != s[r]) return false;
            l++;
            r--;
        }
        return true;
    }

    void solve(int index, string &s, vector<string> &path,
               vector<vector<string>> &ans) {
        if (index == s.size()) {
            ans.push_back(path);
            return;
        }

        for (int i = index; i < s.size(); i++) {
            if (isPalindrome(s, index, i)) {
                path.push_back(s.substr(index, i - index + 1));
                solve(i + 1, s, path, ans);
                path.pop_back();
            }
        }
    }

    vector<vector<string>> partition(string s) {
        vector<vector<string>> ans;
        vector<string> path;
        solve(0, s, path, ans);
        return ans;
    }
};

int main() {
    Solution sol;
    string s = "aab";
```

```
    vector<vector<string>> res = sol.partition(s);

    for (auto &v : res) {
        for (auto &x : v) cout << x << " ";
        cout << endl;
    }
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(2^N * N)`
  All possible partitions are explored, and palindrome checking costs `O(N)`.

- **Space Complexity:** `O(2^N * N) + O(N)`
  For storing all partitions and recursion stack.

19. Word Search (LeetCode)

Explanation
You are given a 2D grid of characters (`board`) and a string `word`.
You need to check whether the word can be formed by **moving through adjacent cells** (up, down, left, right).

Important rules:

- Each character must match the corresponding character in `word`
- You can move only **horizontally or vertically**
- **The same cell cannot be used more than once** in forming the word

---

Example 1

Input:

```
board = [
  ['A','B','C','E'],
  ['S','F','C','S'],
  ['A','D','E','E']
]
word = "ABCCED"
```

Output:

```
true
```

Explanation:
The path `A → B → C → C → E → D` exists in the grid.

Example 2

Input:

```
word = "ABCB"
```

Output:

```
false
```

Explanation:
Although the letters exist, you would need to reuse a cell, which is **not allowed**.

Approach (Backtracking / DFS)

This is a classic **backtracking** problem.

- Start DFS from every cell in the grid
- If the current cell matches the current character of the word:
    - Mark the cell as visited (temporarily)
    - Try moving in all 4 directions
- If at any point the full word is matched → return `true`
- If a path fails, **backtrack** by restoring the cell

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool exist(vector<vector<char>>& board, string word) {
        int rows = board.size();
        int cols = board[0].size();

        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                if (dfs(board, word, i, j, 0))
                    return true;
            }
        }
        return false;
    }

private:
    bool dfs(vector<vector<char>>& board, string &word,
             int i, int j, int idx) {

        if (idx == word.size())
```

```
            return true;

        if (i < 0 || j < 0 || i >= board.size() ||
            j >= board[0].size() || board[i][j] != word[idx])
            return false;

        char temp = board[i][j];
        board[i][j] = '#';  // mark visited

        bool found =
            dfs(board, word, i + 1, j, idx + 1) ||
            dfs(board, word, i - 1, j, idx + 1) ||
            dfs(board, word, i, j + 1, idx + 1) ||
            dfs(board, word, i, j - 1, idx + 1);

        board[i][j] = temp; // backtrack
        return found;
    }
};

int main() {
    Solution sol;
    vector<vector<char>> board = {
        {'A','B','C','E'},
        {'S','F','C','S'},
        {'A','D','E','E'}
    };

    cout << boolalpha << sol.exist(board, "ABCCED") << endl; // true
    cout << boolalpha << sol.exist(board, "SEE") << endl;    // true
    cout << boolalpha << sol.exist(board, "ABCB") << endl;   // false
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(m × n × 4^L)`
  From each cell, we try 4 directions for each character of the word.

- **Space Complexity:** `O(L)`
  Due to recursion stack depth equal to the word length.

20. N-Queen Problem | Return all Distinct Solutions to the N-Queens Puzzle

---

Problem Explanation

You are given an integer **n**, representing an **n × n chessboard**.
Your task is to place **n queens** on the board such that:

1. No two queens attack each other.
2. A queen can attack another queen if they are:
    - In the same **row**
    - In the same **column**
    - On the same **diagonal**

You must return **all distinct valid board configurations**.

Each configuration is represented as:

- `'Q'` → Queen placed
- `'.'` → Empty cell

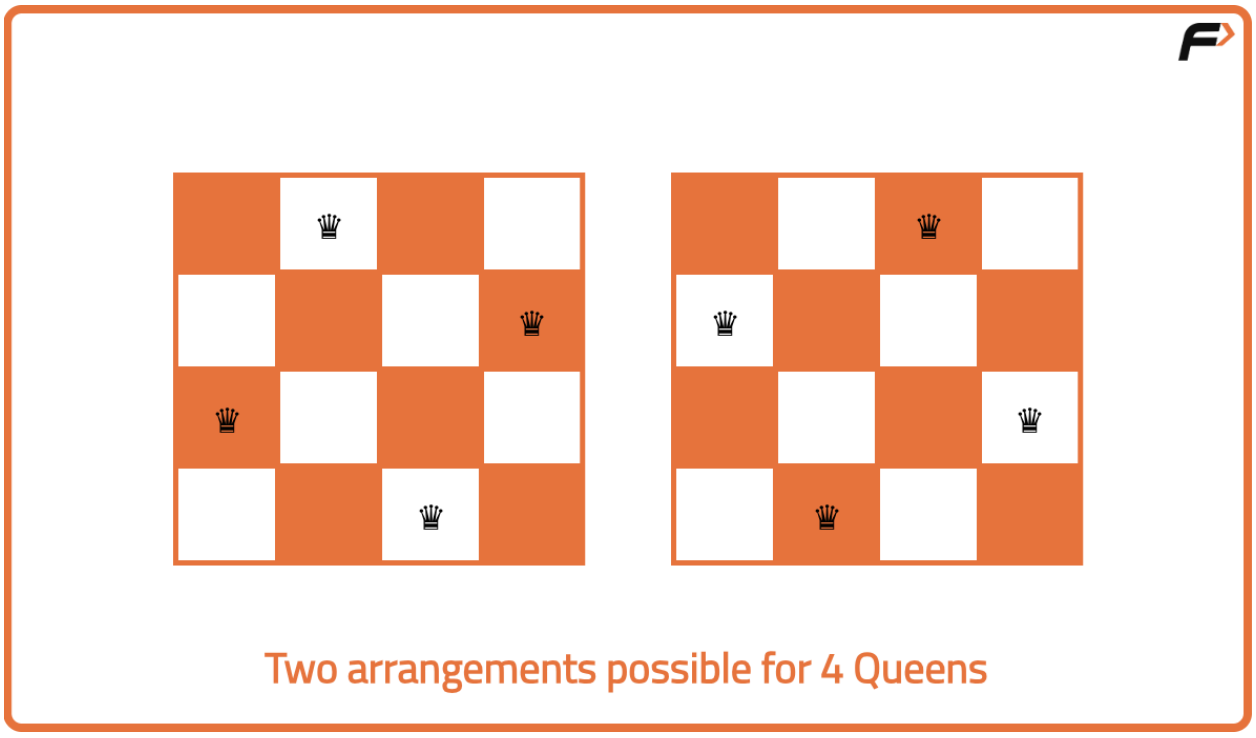
Example 1

Input

```
n = 4
```

Output

```
[
 [".Q..","...Q","Q...","..Q."],
 ["..Q.","Q...","...Q",".Q.."]
]
```



Explanation

There are exactly **2 valid ways** to place 4 queens such that none attack each other.

Example 2

Input

```
n = 1
```

Output

```
[["Q"]]
```

Explanation
Only one cell exists, so placing a single queen is always valid.

Approach 1: Brute Force / Basic Backtracking

Idea

We place queens **column by column**.

For each column:

- Try placing a queen in **every row**
- Before placing, check if it is **safe**
- If safe → place queen and move to next column
- If not → try next row
- If no row works → backtrack

Safety Check (Important)

To check if a queen can be placed at `(row, col)`:

1. Check **same row on the left**
2. Check **upper-left diagonal**
3. Check **lower-left diagonal**

Only left side is checked because we place queens column by column from left to right.

Code (Brute Force)

```
#include <bits/stdc++.h>

using namespace std;


class Solution {

public:

    bool isSafe(int row, int col, vector<vector<char>>& board, int n)
{

        // Check same row (left side)

        for (int j = 0; j < col; j++) {

            if (board[row][j] == 'Q') return false;

        }


        // Check upper-left diagonal

        for (int i = row, j = col; i >= 0 && j >= 0; i--, j--) {

            if (board[i][j] == 'Q') return false;

        }


        // Check lower-left diagonal

        for (int i = row, j = col; i < n && j >= 0; i++, j--) {

            if (board[i][j] == 'Q') return false;
```

```
        }

        return true;
    }

    void solve(int col, vector<vector<char>>& board,
               vector<vector<string>>& ans, int n) {

        if (col == n) {
            vector<string> temp;
            for (int i = 0; i < n; i++) {
                temp.push_back(string(board[i].begin(),
board[i].end()));
            }
            ans.push_back(temp);
            return;
        }

        for (int row = 0; row < n; row++) {
            if (isSafe(row, col, board, n)) {
                board[row][col] = 'Q';
                solve(col + 1, board, ans, n);
                board[row][col] = '.';
```

```
            }
        }
    }

    vector<vector<string>> solveNQueens(int n) {
        vector<vector<string>> ans;
        vector<vector<char>> board(n, vector<char>(n, '.'));
        solve(0, board, ans, n);
        return ans;
    }
};
```

Complexity (Brute Force)

- **Time Complexity:** $O(N! \times N)$
  (Trying all permutations + safety checks)
- **Space Complexity:** $O(N^2)$
  (Board + recursion stack)

Key Idea (Backtracking)

This is a classic **backtracking** problem.

We place queens **column by column**:

- In each column, try placing a queen in every row
- Before placing, check if the position is **safe**
- If safe → place queen and move to next column
- If not → try next row
- If no row works → **backtrack**

Why Backtracking Works Here

- Each queen placement depends on previous placements
- Wrong choices must be undone (backtracking)
- We explore all valid possibilities efficiently

Optimal Approach (Using Hashing for Fast Safety Check)

Instead of checking the entire board every time, we use **3 helper arrays**:

1. `leftRow[row]` → checks if a row already has a queen
2. `lowerDiagonal[row + col]` → checks \ diagonal
3. `upperDiagonal[n - 1 + col - row]` → checks / diagonal

This allows **O(1)** safety checks.

For checking Left now

In the grid, we will fill the sum of indices of row and columns

	0	1	2	3	4	5	6	7
0	0	1	2	3	4	5	6	7
1	1	2	3	4	5	6	7	8
2	2	3	4	5	6	7	8	9
3	3	4	5	6	7	8	9	10
4	4	5	6	7	8	9	10	11
5	5	6	7	8	9	10	11	12
6	6	7	8	9	10	11	12	13
7	7	8	9	10	11	12	13	14

We can check that diagonal elements are same in grid

If we are taking n*n grid we can take maximum value as 2*n-1
for 8*8 grid, maximum value = 2*8-1=15
means hash size is 15



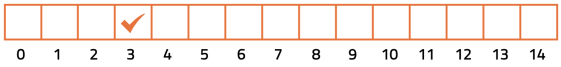
For checking upper diagonal and lower diagonal

In the grid, we will fill the (n-1) + (row-col)

	0	1	2	3	4	5	6	7
0	7	8	9	10	11	12	13	14
1	6	7	8	9	10	11	12	13
2	5	6	7	8	9	10	11	12
3	4	5	6	7	8	9	10	11
4	3	4	5	6	7	8	9	10
5	2	3	4	5	6	7	8	9
6	1	2	3	4	5	6	7	8
7	0	1	2	3	4	5	6	7

We can check that diagonal elements are same in grid

If we are taking n*n grid we can take maximum value as 2*n-1
for 8*8 grid, maximum value = 2*8-1=15
means hash size is 15



For checking Left now

In the grid, we will fill the sum of indices of row and columns

	0	1	2	3	4	5	6	7
0	0	1	2	3	4	5	6	7
1	1	2	3	4	5	6	7	8
2	2	3	4	5	6	7	8	9
3	3	4	5	6	7	8	9	10
4	4	5	6	7	8	9	10	11
5	5	6	7	8	9	10	11	12
6	6	7	8	9	10	11	12	13
7	7	8	9	10	11	12	13	14

We can check that diagonal elements are same in grid

If we are taking n*n grid we can take maximum value as 2*n-1
for 8*8 grid, maximum value = 2*8-1=15
means hash size is 15



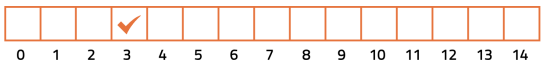
For checking upper diagonal and lower diagonal

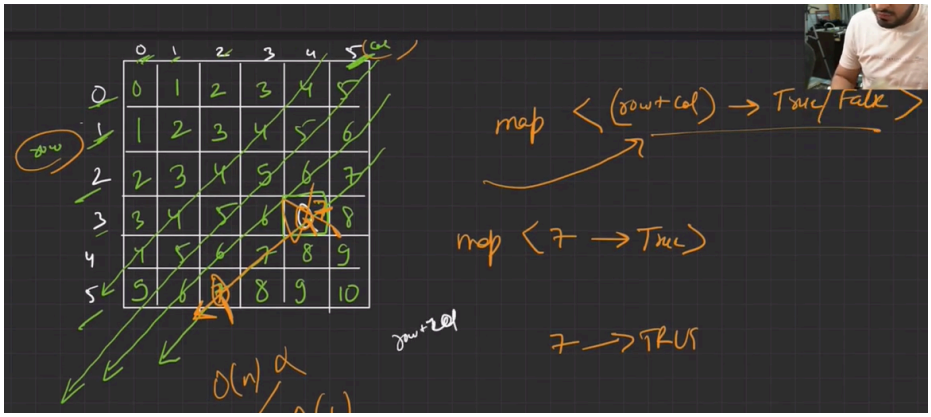
In the grid, we will fill the (n-1) + (row-col)

	0	1	2	3	4	5	6	7
0	7	8	9	10	11	12	13	14
1	6	7	8	9	10	11	12	13
2	5	6	7	8	9	10	11	12
3	4	5	6	7	8	9	10	11
4	3	4	5	6	7	8	9	10
5	2	3	4	5	6	7	8	9
6	1	2	3	4	5	6	7	8
7	0	1	2	3	4	5	6	7

We can check that diagonal elements are same in grid

If we are taking n*n grid we can take maximum value as 2*n-1
for 8*8 grid, maximum value = 2*8-1=15
means hash size is 15





Code (Optimal Solution)

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    void solve(int col, vector<string>& board, int n,
               vector<int>& leftRow,
               vector<int>& upperDiagonal,
               vector<int>& lowerDiagonal,
               vector<vector<string>>& ans) {

        // Base case: all queens placed
        if (col == n) {
            ans.push_back(board);
            return;
        }

        // Try placing queen in every row of this column
        for (int row = 0; row < n; row++) {
            if (leftRow[row] == 0 &&
                lowerDiagonal[row + col] == 0 &&
                upperDiagonal[n - 1 + col - row] == 0) {

                // Place queen
                board[row][col] = 'Q';
                leftRow[row] = 1;
                lowerDiagonal[row + col] = 1;
                upperDiagonal[n - 1 + col - row] = 1;

                // Recurse to next column
                solve(col + 1, board, n, leftRow, upperDiagonal,
lowerDiagonal, ans);

                // Backtrack
                board[row][col] = '.';
                leftRow[row] = 0;
                lowerDiagonal[row + col] = 0;
```

```
                upperDiagonal[n - 1 + col - row] = 0;
            }
        }
    }

    vector<vector<string>> solveNQueens(int n) {
        vector<vector<string>> ans;
        vector<string> board(n, string(n, '.'));

        vector<int> leftRow(n, 0);
        vector<int> lowerDiagonal(2 * n - 1, 0);
        vector<int> upperDiagonal(2 * n - 1, 0);

        solve(0, board, n, leftRow, upperDiagonal, lowerDiagonal,
ans);
        return ans;
    }
};

int main() {
    Solution obj;
    int n = 4;
    vector<vector<string>> res = obj.solveNQueens(n);

    for (auto& board : res) {
        for (auto& row : board) {
            cout << row << "\n";
        }
        cout << "\n";
    }
    return 0;
}
```

Complexity Analysis

Time Complexity

`O(N!)`

- We try all possible permutations of queen placements

Space Complexity

`O(N)`

- Recursion stack + helper arrays

21. Rat in a Maze

Question Explanation

You are given a square grid of size **n × n** consisting of `0`s and `1`s.

- The rat starts at **(0, 0)**
- The destination is **(n-1, n-1)**
- The rat can move in **four directions**:
    - `D` → Down
    - `L` → Left
    - `R` → Right
    - `U` → Up

Rules

1. The rat can move only to cells with value 1
2. A cell with value `0` is blocked
3. A cell cannot be visited more than once in the same path
4. If the starting cell `(0,0)` is `0`, no path exists
5. You must return **all possible paths** in **lexicographical order**

Example 1

Input

```
n = 4
grid =
[
 [1, 0, 0, 0],
 [1, 1, 0, 1],
 [1, 1, 0, 0],
 [0, 1, 1, 1]
]
```

Output

```
["DDRDRR", "DRDDRR"]
```

Explanation

Two valid paths exist from `(0,0)` to `(3,3)` following the movement rules.

Example 2

Input

```
n = 2
grid =
[
 [1, 0],
 [1, 0]
]
```

Output

```
[]
```

Explanation

The destination cell $(1, 1)$ is blocked, so no valid path exists.

Approach: Backtracking (DFS)

Intuition

Since we need to find **all possible paths**, the rat must explore the maze **step by step**, trying every valid direction.

If the rat:

- Hits a blocked cell
- Goes out of bounds
- Revisits a cell

→ that path is abandoned (backtracking).

This guarantees:

- No infinite loops
- Every valid path is explored exactly once

Key Points

- Use a `visited` matrix to avoid revisiting cells

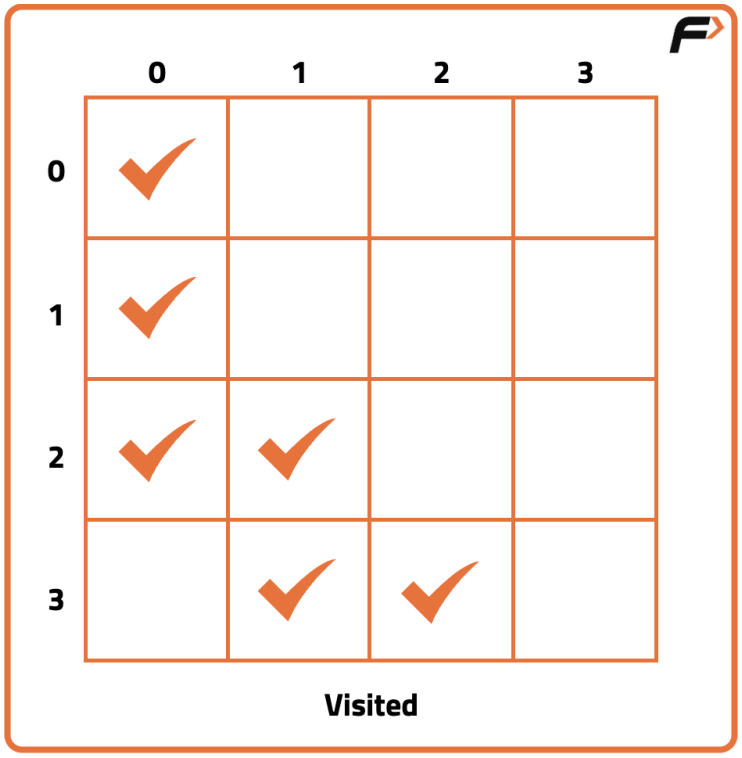
Always explore directions in **lexicographical order**:

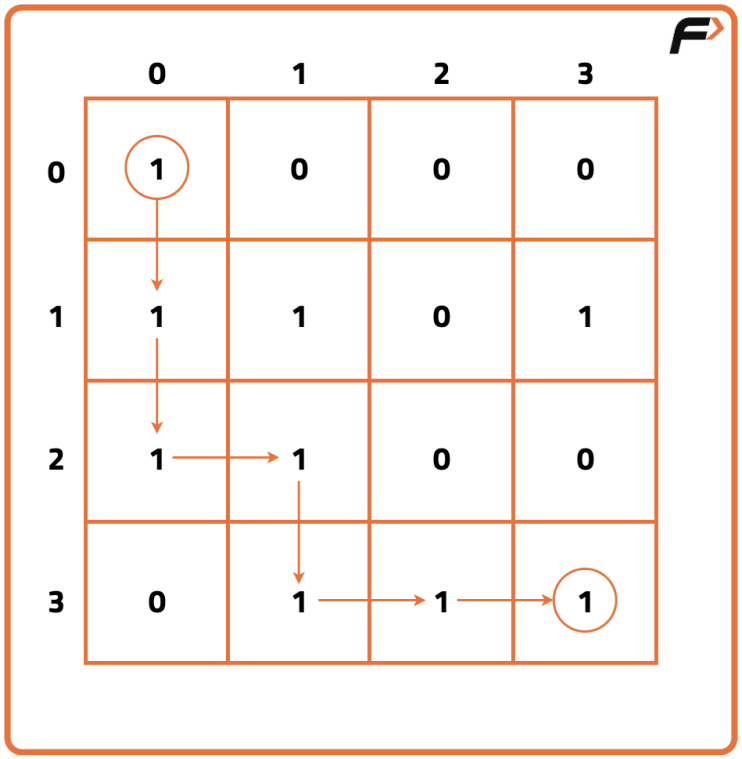
`D → L → R → U`

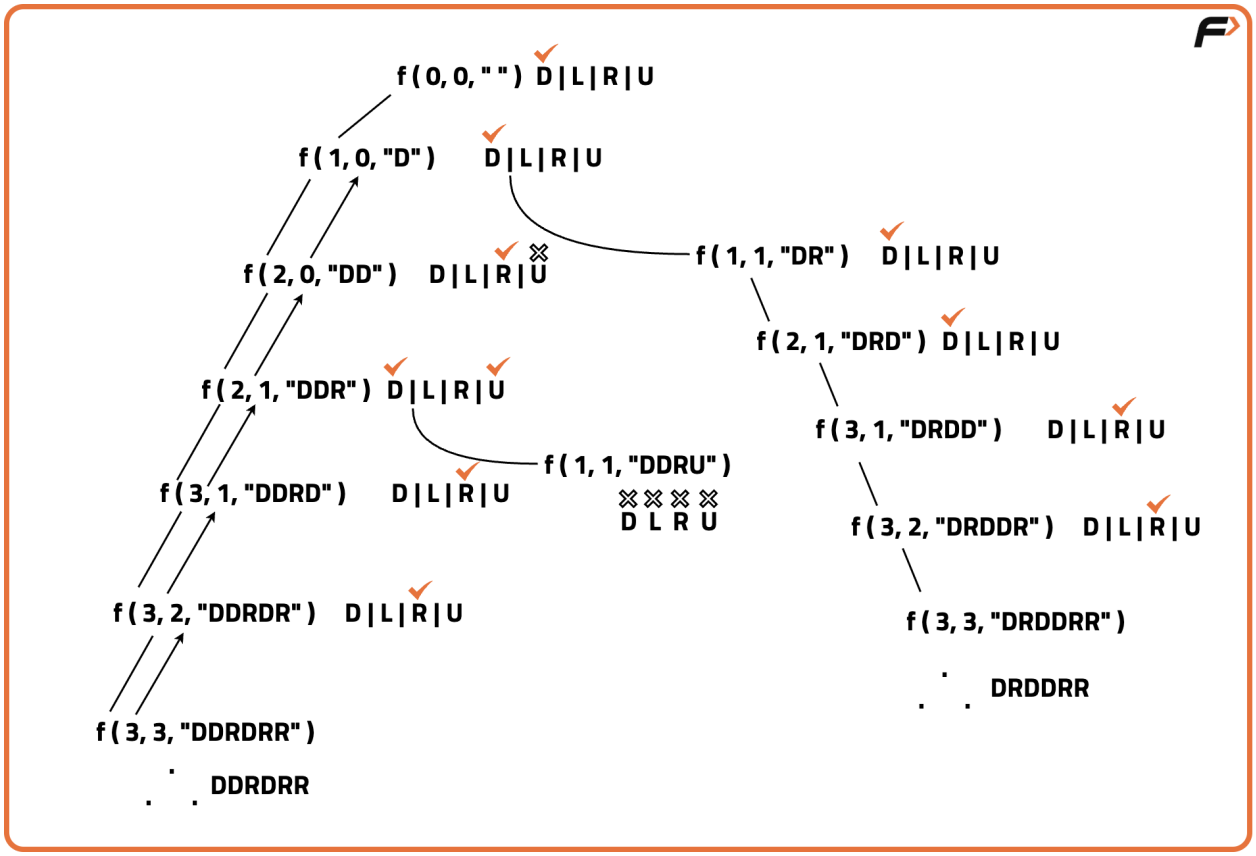
-
- Mark a cell as visited when moving forward
- Unmark it while backtracking

Algorithm Steps

1. If `maze[0][0] == 0`, return empty answer
2. Start DFS from `(0,0)` with an empty path string
3. At each cell:
    - Try all 4 directions in order: `D, L, R, U`
4. If destination `(n-1, n-1)` is reached:
    - Store the current path
5. Backtrack by unmarking the visited cell


0 1 2 3
0
1
2
3
Visited


0 1 2 3
0
1
2
3
1 0 0 0
1 1 0 1
1 1 0 0
0 1 1 1



Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    // Check if the move is valid
    bool isSafe(int x, int y, int n,
                vector<vector<int>>& maze,
                vector<vector<int>>& visited) {
        return (x >= 0 && x < n &&
                y >= 0 && y < n &&
                maze[x][y] == 1 &&
                visited[x][y] == 0);
    }

    // Backtracking function
```

```
void solve(int x, int y, int n,
           vector<vector<int>>& maze,
           vector<vector<int>>& visited,
           string path,
           vector<string>& res) {

    // Destination reached
    if (x == n - 1 && y == n - 1) {
        res.push_back(path);
        return;
    }

    visited[x][y] = 1;

    // Down
    if (isSafe(x + 1, y, n, maze, visited))
        solve(x + 1, y, n, maze, visited, path + "D", res);

    // Left
    if (isSafe(x, y - 1, n, maze, visited))
        solve(x, y - 1, n, maze, visited, path + "L", res);

    // Right
    if (isSafe(x, y + 1, n, maze, visited))
        solve(x, y + 1, n, maze, visited, path + "R", res);

    // Up
    if (isSafe(x - 1, y, n, maze, visited))
        solve(x - 1, y, n, maze, visited, path + "U", res);

    // Backtrack
    visited[x][y] = 0;
}

vector<string> findPath(vector<vector<int>>& maze, int n) {
    vector<string> res;
    vector<vector<int>> visited(n, vector<int>(n, 0));
```

```
        if (maze[0][0] == 1) {
            solve(0, 0, n, maze, visited, "", res);
        }

        return res;
    }
};
```

Complexity Analysis

- **Time Complexity:** `O(4^(n*n))`
  In the worst case, the rat explores all 4 directions from every cell.

- **Space Complexity:** `O(n*n)`
  For the visited matrix and recursion stack.

22. Word Break

Question Explanation

You are given:

- A string **s**

- A dictionary **wordDict** containing valid words

Your task is to check **whether the string s can be broken into a sequence of dictionary words**.

Important points

- You can reuse dictionary words multiple times
- The order must follow the original string
- Every part of the string must belong to the dictionary
- If even one segment is invalid → return `false`

Example 1

Input

```
s = "takeuforward"
wordDict = ["take", "forward", "you", "u"]
```

Output

```
true
```

Explanation

The string can be segmented as:

```
"take" + "u" + "forward"
```

All parts exist in the dictionary.

Example 2

Input

```
s = "applepineapple"
wordDict = ["apple"]
```

Output

`false`

Explanation

Possible split:

`"apple" + "pine" + "apple"`

But `"pine"` is **not** in the dictionary, so segmentation fails.

Approach: Dynamic Programming

Intuition

At every index of the string, we ask:

> **Can the substring up to this index be segmented using dictionary words?**

If yes, we mark it as valid and continue forward.

We use DP because:

- The same substrings are checked repeatedly
- DP avoids recomputation
- Efficient for large strings

DP Definition

Let:

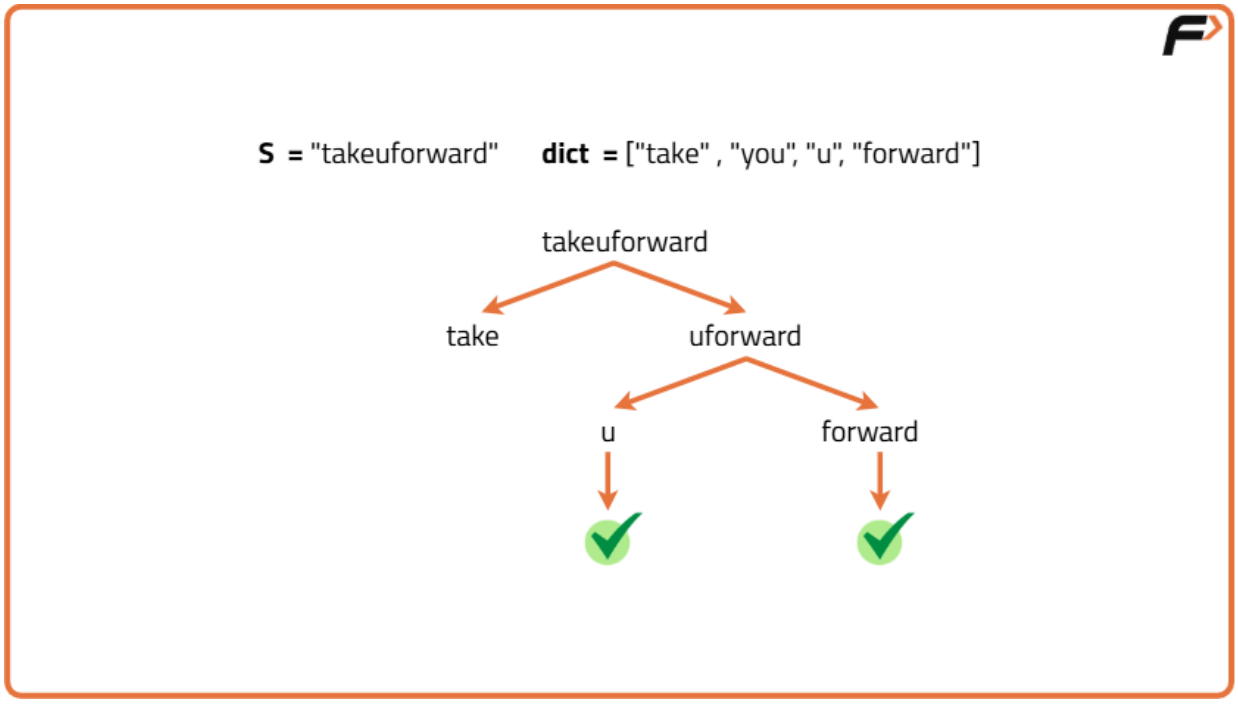
`dp[i] = true`

if substring `s[0 … i-1]` **can be segmented**

Final answer will be:

`dp[n]`

where `n = length of s`



Algorithm (Step-by-Step)

1. Convert `wordDict` into a **set** for fast lookup
2. Create a boolean array dp of size `n + 1`

Initialize:

`dp[0] = true`

3. (empty string is always valid)
4. Find the **maximum word length** in dictionary
    - This limits unnecessary substring checks
5. For every position `i` from 1 to n:
    - Try splitting at position `j`

If:

`dp[j] == true`
`AND s[j … i-1] exists in dictionary`
then mark:

`dp[i] = true`

    -
6. Return `dp[n]`

Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool wordBreak(string s, vector<string>& wordDict) {
        int n = s.length();

        // Store dictionary words in a set for O(1) lookup
        unordered_set<string> dict(wordDict.begin(), wordDict.end());
```

```
        // dp[i] = true if s[0..i-1] can be segmented
        vector<bool> dp(n + 1, false);
        dp[0] = true;  // empty string

        // Find maximum length of any word in dictionary
        int maxLen = 0;
        for (string &w : wordDict) {
            maxLen = max(maxLen, (int)w.size());
        }

        // Fill dp array
        for (int i = 1; i <= n; i++) {
            for (int j = max(0, i - maxLen); j < i; j++) {
                if (dp[j] && dict.count(s.substr(j, i - j))) {
                    dp[i] = true;
                    break;
                }
            }
        }

        return dp[n];
    }
};
```

Complexity Analysis

Time Complexity

`O(n * m)`

- `n` = length of string
- `m` = maximum word length
- Limited substring checks improve performance

Space Complexity

`O(n)`

- DP array of size `n + 1`
- Dictionary set

23. M – Coloring Problem

Question Explanation

You are given:

- An **undirected graph** with `N` vertices
- A number `M` representing available colors

Your task is to check **whether it is possible to color all vertices** such that:

- Each vertex has **one color**
- **No two adjacent vertices** have the **same color**
- You can use **at most M colors**

Return:

- `1` → if coloring is possible
- `0` → if coloring is not possible

Example 1

Input

```
N = 4
M = 3
Edges = {(0,1), (1,2), (2,3), (3,0), (0,2)}
```

Output

```
1
```

Explanation

The graph can be colored using 3 colors without any adjacent vertices sharing the same color.

Example 2

Input

```
N = 3
M = 2
Edges = {(0,1), (1,2), (0,2)}
```

Output

```
0
```

Explanation

The graph forms a triangle.
At least **3 colors** are needed, so coloring with 2 colors is not possible.

Approach (Backtracking)

Intuition

We try to assign colors to vertices **one by one**.

For each vertex:

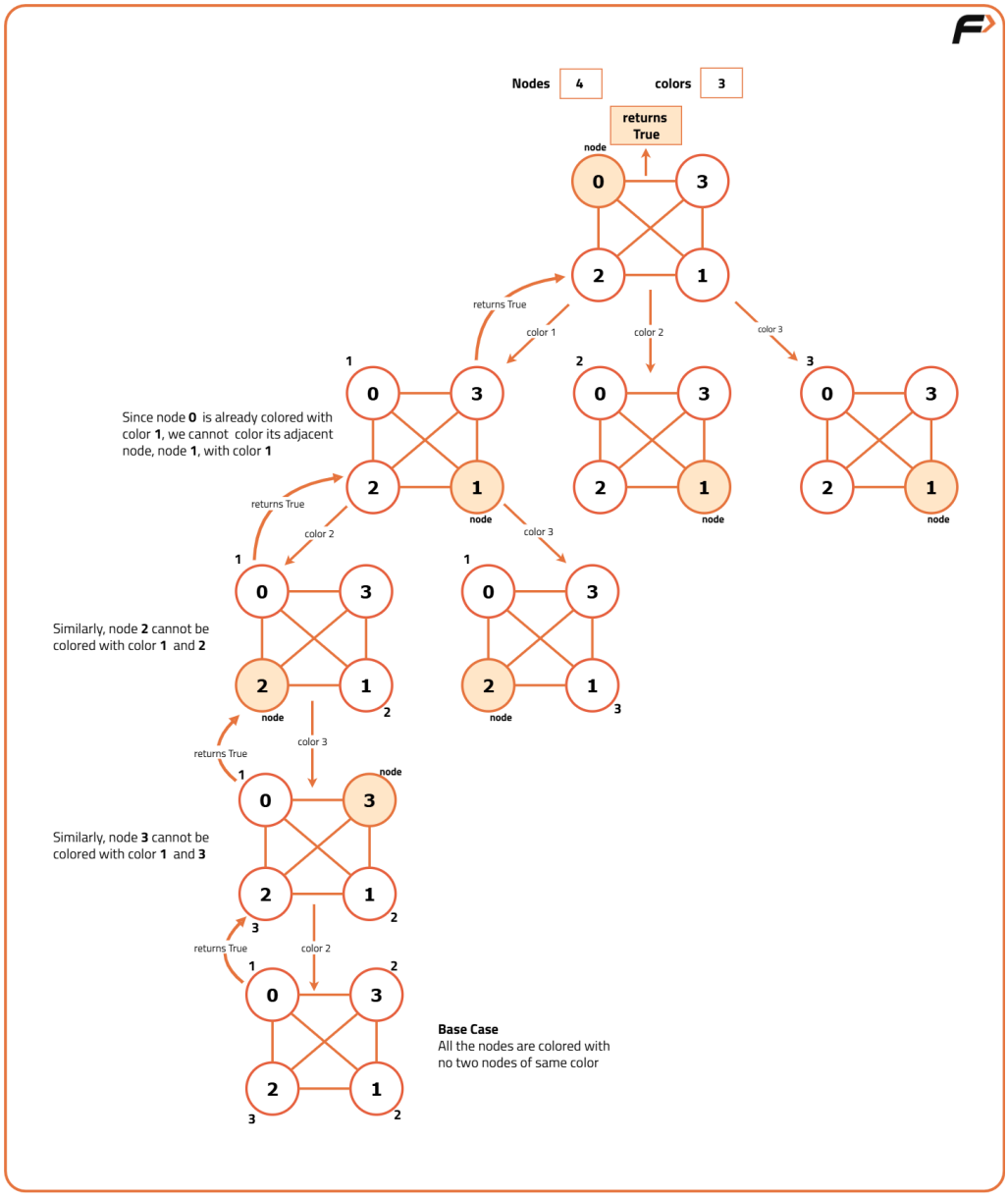
- Try every color from 1 to `M`
- Check if assigning that color is **safe**
- If safe → move to the next vertex
- If not → try another color
- If all colors fail → backtrack

This explores all possible color assignments.

Algorithm (Step-by-Step)

1. Start coloring from vertex `0`
2. For the current vertex:
    - Try each color from 1 to `M`
3. Use `isSafe()` to check:
    - No adjacent vertex has the same color
4. If safe:
    - Assign the color
    - Recursively color the next vertex

5. If recursion fails:
    - Remove the color (backtrack)
6. If all vertices are colored successfully:
    - Return `true`
7. If no color works:
    - Return `false`



Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

// Check if it is safe to color the node with given color
bool isSafe(int node, int color[], bool graph[101][101], int n, int
col) {
    for (int k = 0; k < n; k++) {
        if (graph[node][k] && color[k] == col) {
            return false;
        }
    }
    return true;
}

// Backtracking function
bool solve(int node, int color[], int m, int n, bool graph[101][101])
{
    // All nodes are colored
    if (node == n) return true;

    // Try all colors
    for (int col = 1; col <= m; col++) {
        if (isSafe(node, color, graph, n, col)) {
            color[node] = col;

            if (solve(node + 1, color, m, n, graph))
                return true;

            // Backtrack
            color[node] = 0;
        }
    }
    return false;
}
```

```
// Main function
bool graphColoring(bool graph[101][101], int m, int n) {
    int color[n];
    memset(color, 0, sizeof(color));

    return solve(0, color, m, n, graph);
}

int main() {
    int n = 4, m = 3;
    bool graph[101][101] = {0};

    graph[0][1] = graph[1][0] = 1;
    graph[1][2] = graph[2][1] = 1;
    graph[2][3] = graph[3][2] = 1;
    graph[3][0] = graph[0][3] = 1;
    graph[0][2] = graph[2][0] = 1;

    cout << graphColoring(graph, m, n);
    return 0;
}
```

Complexity Analysis

Time Complexity

`O(M^N)`

- For each vertex, we try `M` colors

Space Complexity

`O(N)`

- Color array

- Recursion stack

24. Sudoku Solver

Question Explanation

You are given a **9×9 Sudoku board** where some cells are empty and represented by `'.'`.

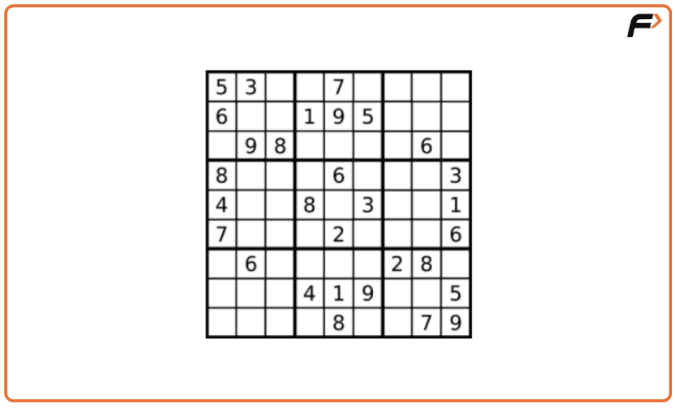
Your task is to **fill all empty cells** such that the final board satisfies **Sudoku rules**:

1. Every row must contain digits 1 to 9 exactly once
2. Every column must contain digits 1 to 9 exactly once
3. Each 3×3 sub-box must contain digits 1 to 9 exactly once

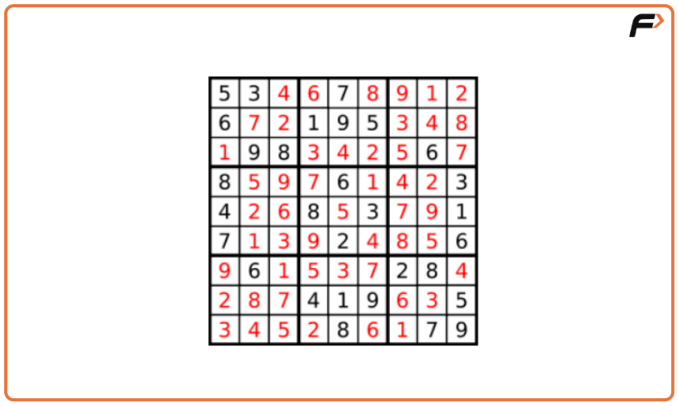
You must modify the board **in-place** and produce **any one valid solution**.

Example (Conceptual)

Input board contains empty cells `.`



Output board is fully filled and valid according to Sudoku rules.



5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

Approach (Backtracking – as given)

Intuition

Sudoku is a **trial-and-error** problem.

Whenever we find an empty cell:

- Try placing digits from 1 to 9
- Check if the placement is valid
- If valid → move forward recursively
- If no number works → backtrack

This guarantees that all possibilities are explored safely.

Algorithm (Step-by-Step)

1. Traverse the board cell by cell
2. When an empty cell `'.'` is found:
    - Try digits from `'1'` to `'9'`

3. For each digit, check validity using `isValid()`:
    - Row check
    - Column check
    - 3×3 sub-box check
4. If valid:
    - Place the digit
    - Recursively solve the rest of the board
5. If recursion fails:
    - Remove the digit (backtrack)
6. If all cells are filled successfully:
    - Return true

Validity Check Logic

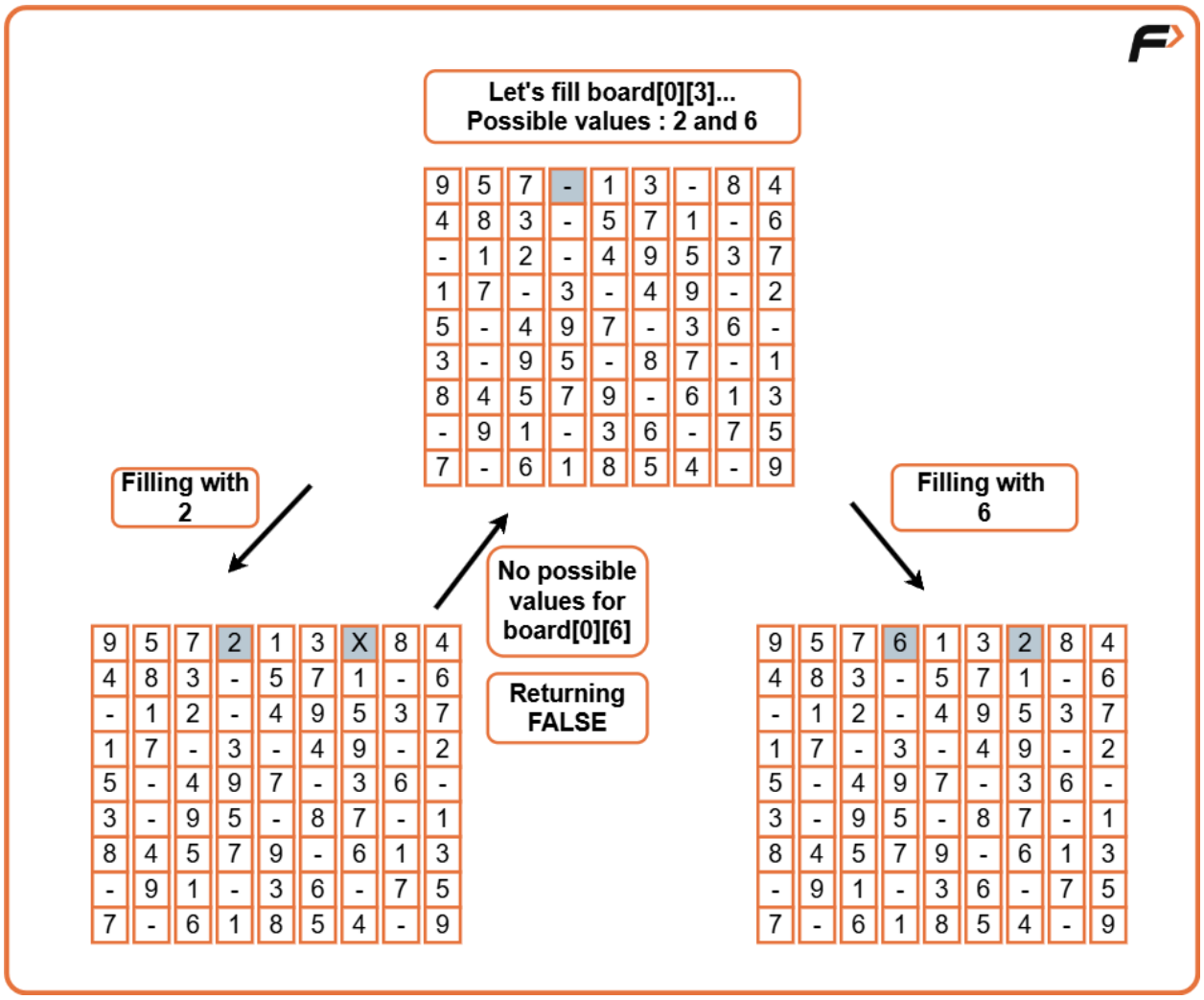
For placing digit c at `(row, col)`:

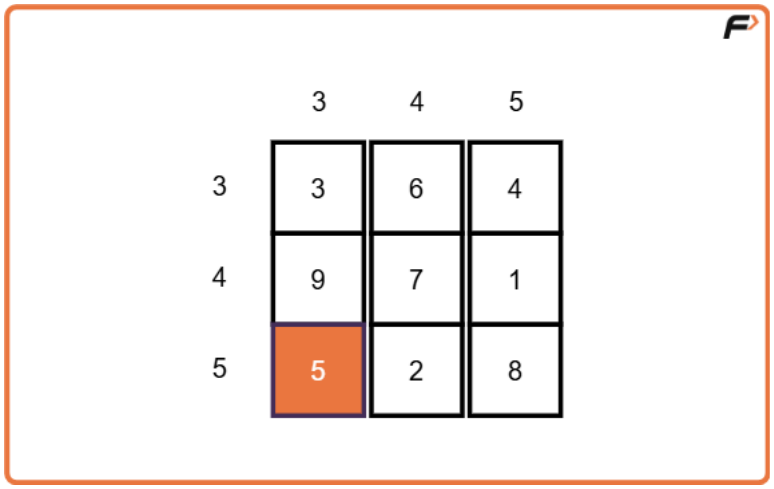
- Check column → `board[i][col]`
- Check row → `board[row][i]`

Check 3×3 box using:

```
 rowIndex = 3 * (row / 3) + i / 3
colIndex = 3 * (col / 3) + i % 3
```

-


Let's fill board[0][3]...
Possible values : 2 and 6
Filling with 2
Filling with 6
No possible values for board[0][6]
Returning FALSE


3
4
5
3
4
5
3 6 4
9 7 1
5 2 8

Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    // Check if placing character c at board[row][col] is valid
    bool isValid(vector<vector<char>> &board, int row, int col, char
c) {
        for (int i = 0; i < 9; i++) {
            if (board[i][col] == c) return false;
            if (board[row][i] == c) return false;
            if (board[3 * (row / 3) + i / 3]
                     [3 * (col / 3) + i % 3] == c) //integer division
                return false;
        }
        return true;
    }

    // Backtracking solver
    bool solveSudoku(vector<vector<char>> &board) {
        for (int i = 0; i < 9; i++) {
            for (int j = 0; j < 9; j++) {
                if (board[i][j] == '.') {
                    for (char c = '1'; c <= '9'; c++) {
                        if (isValid(board, i, j, c)) {
                            board[i][j] = c;
                            if (solveSudoku(board))
                                return true;
                            board[i][j] = '.';
                        }
                    }
                    return false;
                }
            }
        }
        return true;
    }
```

```
};

int main() {
    vector<vector<char>> board {
        {'9','5','7','.','1','3','.','8','4'},
        {'4','8','3','.','5','7','1','.','6'},
        {'.','1','2','.','4','9','5','3','7'},
        {'1','7','.','3','.','4','9','.','2'},
        {'5','.','4','9','7','.','3','6','.'},
        {'3','.','9','5','.','8','7','.','1'},
        {'8','4','5','7','9','.','6','1','3'},
        {'.','9','1','.','3','6','.','7','5'},
        {'7','.','6','1','8','5','4','.','9'}
    };

    Solution sol;
    sol.solveSudoku(board);

    for (int i = 0; i < 9; i++) {
        for (int j = 0; j < 9; j++)
            cout << board[i][j] << " ";
        cout << "\n";
    }
    return 0;
}
```

Complexity Analysis

Time Complexity

O(9^(n^2))

- For each empty cell, we try up to 9 numbers

Space Complexity

O(1)

- Board is modified in-place
- Only recursion stack is used

25. Expression Add Operators

Question Explanation

You are given:

- A string `num` consisting only of digits
- An integer `target`

Your task is to **insert binary operators** (`+`, `-`, `*`) **between the digits** of `num` such that:

- The resulting mathematical expression evaluates exactly to `target`
- Operands must **not contain leading zeros** (except the number `"0"` itself)
- Digits must remain in the same order

You must return **all valid expressions**.

Example

Example 1

Input

```
num = "123", target = 6
```

Output

```
["1*2*3", "1+2+3"]
```

Example 2

Input

```
num = "232", target = 8
```

Output

```
["2*3+2", "2+3*2"]
```

Approach (DFS + Backtracking)

Core Idea

We generate **all possible expressions** by:

- Splitting the string into numbers
- Inserting operators between them
- Evaluating the expression **on the fly** using recursion

This avoids building and re-evaluating expressions repeatedly.

Important Observations

1. **Leading Zero Rule**
    - `"0"` is allowed

- "05", "00" are NOT allowed
  → If a number starts with '0' and has more than one digit, skip it.

2. **Operator Precedence**
   - * has higher precedence than + and -
   - To handle this, we track:
     - current_value: total value so far
     - last_operand: last number added (used to fix multiplication)

Recursive DFS Parameters

Parameter	Meaning
start	Current index in string
expression	Expression built so far
current_value	Evaluated value till now
last_operand	Last operand used (for *)

Base Case

When start == num.length():

- If current_value == target
- Store the expression

Recursive Choices

For each substring starting at `start`:

1. Take substring as a number
2. If first number → start expression
3. Else try:
    - `+`
    - `-`
    - `*` (adjust using `last_operand`)

Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    void dfs(string& num, int target, int start,
             long long current_value, long long last_operand,
             string expression, vector<string>& result) {

        // Base case
        if (start == num.size()) {
            if (current_value == target)
                result.push_back(expression);
            return;
        }

        // Try all possible numbers starting at 'start'
        for (int i = start; i < num.size(); i++) {

            // Leading zero check
```

```
            if (i > start && num[start] == '0') return;

            string part = num.substr(start, i - start + 1);
            long long value = stoll(part);
```

(In C++, **std::stol()** and **std::stoll()** are the library functions used to convert the given string to integer value of type long int and long long int respectively)

```
            if (start == 0) {
                // First number, no operator
                dfs(num, target, i + 1, value, value, part, result);
            } else {
                // Addition
                dfs(num, target, i + 1,
                    current_value + value, value,
                    expression + "+" + part, result);

                // Subtraction
                dfs(num, target, i + 1,
                    current_value - value, -value,
                    expression + "-" + part, result);

                // Multiplication
                dfs(num, target, i + 1,
                    current_value - last_operand + last_operand *
value,
                    last_operand * value,
                    expression + "*" + part, result);
            }
        }
    }

    vector<string> addOperators(string num, int target) {
        vector<string> result;
        dfs(num, target, 0, 0, 0, "", result);
        return result;
    }
};

int main() {
    string num = "123";
```

```
    int target = 6;

    Solution sol;
    vector<string> ans = sol.addOperators(num, target);

    for (auto &s : ans)
        cout << s << " ";

    return 0;
}
```

Complexity Analysis

Time Complexity

`O(4^n)`

- At each position, we explore multiple splits and operators

Space Complexity

`O(n)`

- Recursion depth up to string length
- Expression string built incrementally